



JOMO KENYATTA UNIVERSITY OF AGRICULTURE & TECHNOLOGY

**SCHOOL OF OPEN, DISTANCE &
eLEARNING**

**IN COLLABORATION WITH
SCHOOL OF COMPUTING AND
INFORMATION TECHNOLOGY**

**DEPARTMENT OF INFORMATION
TECHNOLOGY**

BIT 2114 OBJECT-ORIENTED ANALYSIS AND DESIGN

LAST REVISION ON April 30, 2014

P. O. OYIER

(oyier@itc.jkuat.ac.ke)

P.O. Box 62000, 00200

Nairobi, Kenya

BIT 2114: OBJECT-ORIENTED ANALYSIS AND DESIGN

Course Description

Introduction; The inherent Complexity of software, The Structure of Complex Systems, Categories of Analysis and Design Methods. The Evolution of the Object Model, Elements of the Object Model, Applying the Object Model, Foundations of the Object Model. The Nature of an Object, Relationships among Objects. The Nature of a Class, Relationships Among Classes. The Interplay of Classes and Objects, On Building Quality Classes and Objects, Invoking a Method. The importance of proper classification, Identifying Classes and Objects, Key Abstraction and Mechanism, A Problem of Classification. The Notation and the Process; Elements of the Notation, Class Diagram. State Transition Diagrams, Object Diagrams, Intersection Diagram, Module Diagrams, Process Diagrams, Applying the Notation, First Principles, The Micro Development process, The Macro Development process. Pragmatics; Management and Planning, Staffing, Release Management, Reuse, Quality Assurance and Metrics, Documentation, Tools, Domain specific issues, Technology Transfer. The Benefits and Risk of Object-Oriented Development. Applications; Data Acquisition: Weather Monitoring System, Frameworks: Foundation Class Library, Client Server Computing: Inventory Tracking, Command and Control: Traffic Management.

Prerequisite: BIT 2112 Systems Analysis and Design

Course aims

Students will learn how to produce detailed object models and designs from system requirements; use the modeling concepts provided by UML; identify use cases and expand them into full behavioral designs; expand the analysis into a design ready for implementation and construct designs that are reliable.

Learning outcomes

Upon completion of this course you should be able to;

1. Demonstrate sound understanding of the fundamental concepts of the object model.

-
2. Discuss how large, complex software systems are developed using modern software engineering methods and models.
 3. Describe basic OO analysis and design skills through elaborate case studies
 4. Use and apply the appropriate UML design diagrams
 5. Demonstrate the realistic application of object oriented development within a variety of problem domains.

Instruction methodology

Lectures, practical and tutorial sessions in Computer Laboratory, individual and group assignments, exercises and project work.

Course Text Books

1. Grady Booch (2007). Object oriented analysis and Design with applications, ISBN-13: 978-0201895513
2. J W. Satzinger, Robert B.Jackson, Stephen D. Burd (2004). Object oriented analysis and Design with Unified Process, ISBN-13: 978-0619216436
3. Edward Coad (1990). Object Oriented Analysis, ISBN-13: 978-0136299813

Reference Text books

1. Grady Booch, James Rumbaugh, Ivar Jacobson (2005). The Unified Modeling Language User Guide, ISBN-13: 978-0321267979
2. Richard Lee , William M.Tepfenhart (2000). UML and C++ - A practical guide to object oriented development” , ISBN-13: 978-0130290403
3. John. W Satzinger (2004). Object Oriented Analysis and Design with the Unified Process. Cengage Learning, ISBN-13: 978-0619216436

Course Journals

1. Acta Informatica ISSN 0001-5903
2. Advances in Computational Mathematics ISSN 1019-7168

-
3. Advances in data Analysis and Classification ISSN1 1862-5347
 4. Annals Of software Engineering ISSN 1022-7091

Reference Journals

1. Journal of computer science and Technology ISSN 1000-9000
2. Journal of Science and Technology ISSN 1860-4749
3. Central European Journal Of Computer Science ISSN 1896-1533
4. Cluster computing ISSN 1386-7857

Assessment information

The module will be assessed as follows;

- 10% of marks from two (2) assignments
- 20% of marks from one written CAT to be administered at JKUAT main campus or one of the approved centres
- 70% of marks from written Examination to be administered at JKUAT main campus or one of the approved centres

Contents

1	Object-Oriented Analysis and Design	1
1.1	Introduction	1
1.1.1	Review of Approaches to System Development	1
1.1.2	The Evolution of Systems Development Methodologies	3
1.1.3	Object Oriented Analysis and Design	8
1.1.4	Computing as Simulation	11
1.1.5	Criticisms of Object Oriented Analysis and Design	12
1.1.6	Benefits of Object-oriented Approach	13
1.2	Summary	13
2	Information System Development	16
2.0.1	Problems in Information Systems Development	17
2.0.2	Why Things Go Wrong	20
2.0.3	Avoiding The Problems	21
2.0.4	Project Life Cycles	21
	• Traditional Life Cycle	21
	• Prototyping	21
	• Incremental Development	22
	• Unified Software Development Process	22
2.0.5	Managing Information Systems Development	23
3	Identifying and Defining Objects and Classes	30
3.1	Objects and Classes	31
3.2	Analysis - The Rudiments of an Approach	32
3.2.1	Removing Synonyms and/or Generalizing	32
3.2.2	Look for Attributes	33
3.2.3	Irrelevant Objects	33

3.3	The Process of Development	34
3.4	Object Models	43
4	Requirements Capture	47
4.1	Requirements Capture	48
4.2	Sources of Requirements	49
4.2.1	Stakeholder Identification	50
4.2.2	The Elicitation Process	51
4.2.3	Starting Points for Elicitation	52
4.2.4	Where do we Start?	53
4.2.5	Starting Point	53
4.3	Problems Faced While Eliciting Requirements.	55
5	UML Diagrams	60
5.1	The Unified Modeling Language (UML)	61
5.2	System's Behavior from User's Point of View: Use Case Diagrams	64
5.2.1	Use Cases for Requirements Capture	65
5.2.2	Use Cases and Scenarios	65
5.2.3	Specifying a Use Case	66
5.2.4	Use Case Diagrams	67
5.2.5	What is an Actor?	67
5.3	Associations Between Use Cases	70
5.4	Summary	74
6	Requirements Analysis	81
6.1	Requirements Analysis	82
6.1.1	Requirements Classification	82
6.1.2	Functional Requirements	82
6.1.3	Non- Functional Requirements.	84
6.2	Summary	90
7	Domain Modeling	93
7.1	UML Domain Modeling	94
7.2	Class Diagram	94
7.2.1	Adding Attributes	95

7.2.2	Operations	95
7.3	Class Diagram Relationships	96
7.3.1	Class Diagram: Generalization	97
7.3.2	Groups	98
7.3.3	Class Diagram: Associations	100
7.3.4	Class Diagram: Composition	102
7.3.5	Class Diagram: Aggregation	103
7.4	Example	105
7.5	Class Stereotypes	106
7.5.1	Boundary	107
7.5.2	Entity Classes	108
7.5.3	Control Classes	109
7.6	Summary	110
8	Interaction Diagrams	113
8.1	Interaction Diagrams	114
8.2	Sequence Diagram	114
8.2.1	Elements of a Sequence Diagram (Notations/Symbols) . . .	115
8.2.2	Message Labeling	117
8.3	Summary	123
8.4	Collaboration Diagram	123
9	Other UML Diagrams	127
9.1	Development of Dynamic Behavior	128
9.1.1	Dynamic Behavior: Perspective	128
9.1.2	Use Cases and Interaction Diagrams	128
9.1.3	Use Case and System Interaction	129
9.1.4	Contracts	129
9.1.5	Contracts and Collaboration Diagrams	129
9.2	State Transition Diagram	129
9.2.1	Components of State Diagram	130
9.2.2	Semantics of Every Component	130
9.2.3	Notations	132
9.2.4	Steps for Building State Chart Diagram	133
9.2.5	State Transition Diagram for Account Class	134

9.3	Activity Diagrams	134
9.3.1	Consistency Checking	135
9.4	Implementation Diagrams	136
9.5	Component Diagram	136
9.5.1	Notation	136
9.5.2	Component Diagram for Withdrawal of Cash	137
9.6	Deployment Diagram	137
9.6.1	Notation	137
9.7	Package Diagrams	138
9.8	Summary	138
10	Production of Specification Document	141
10.1	Architecture and the UML	142
10.1.1	Goals of the UML	142
10.1.2	Object-Oriented Developing Process	145
10.1.3	Steps Used in UML	145
10.1.4	Difficulties of Object-Oriented Design	146
10.2	From UML to the Unified Process	146
10.2.1	Creating the Unified Process	147
10.2.2	What Is a Process?	147
10.2.3	Lifecycle Phases	147
10.2.4	Unified Process Structure Iterations and Workflow	148
10.2.5	Workflows and Models	148
10.2.6	Use Case Model	149
10.2.7	Analysis and Design Model	149
10.2.8	Deployment and Implementation Model	150
10.2.9	Test Model	150
10.2.10	Use Case Driven	151
10.3	Production of Specification Document	151
10.3.1	Document Outline	151
10.4	Summary	153

LESSON 1

Object-Oriented Analysis and Design

Learning outcomes

Upon completing this topic, you should be able to:

1. Define the terms used in object orientation.
2. Outline the importance of object oriented analysis and design phase.
3. Describe attributes of objects.
4. Explain usefulness of Object oriented concepts such as abstraction, modularity and Inheritance.

1.1. Introduction

1.1.1. Review of Approaches to System Development

There are many ways to develop an information system. Developers rely on many concepts for help, including methodologies, models, tools, and techniques.

Methodologies, Models, Tools, and Techniques

1. System Development Methodology
 - A system development methodology Provides guidelines to follow for completing every activity in the system development life cycle, including specific models, tools, techniques, and a system development life cycle.
 - There are many kind of methodologies
1. Models
 - A representation of some important aspect of the real world
 - Some models used in system development
 - Flow chart
 - Data flow diagram (DFD)

- Entity-relationship diagram (ERD)
 - Structure chart
 - Use case diagram
 - Class diagram
 - Sequence diagram
- Some models used to manage the development process
 - PERT chart
 - Gantt chart
 - Organizational hierarchy chart
 - Financial analysis models - NPV, ROI

1. Tools

- Software support that helps create models or other components required in the project.
- Some tools used in system development
 - Project management application
 - Drawing/graphics application
 - Word processor/text editor
 - CASE tools (Computer-aided system engineering tool designed to help a systems analyst complete development tasks)
 - Integrated development environment (IDE)
 - Database management application
 - Reverse-engineering tool
 - Code generator tool

1. Techniques

- A collection of guidelines that help the analyst complete a system development activity or task.

- Some techniques used in system development
 - Strategic planning techniques
 - Project management techniques
 - User interviewing techniques
 - Data-modeling techniques
 - Relational database design techniques
 - Structured analysis technique
 - Structured design technique
 - Structured programming techniques
 - Software-testing techniques
 - Object-oriented analysis and design techniques

Relationship Between Components of a Methodology

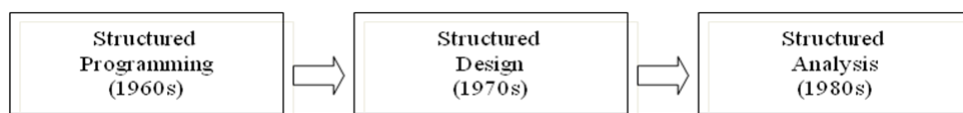


Figure 1.1: Relationship between Methodology, Techniques and Models

1.1.2. The Evolution of Systems Development Methodologies

1. Systems Development Life Cycle
2. The Structured Analysis and Design Approach
3. The Information Engineering Approach
4. The Object-Oriented Approach

Systems Development Life Cycle (SDLC)

- Considers systems development as a single cycle of sequential steps
- There might be other cycles during the operation/maintenance of the system
- Many variations of SDLC in practice

- Based on variation of names for phases
 - No matter which one, activities/tasks are similar
- Some increase emphasis on people
 - User-centered design, participatory design
 - Sociotechnical systems
- Some increase speed of development
 - Rapid application development (RAD)
 - Prototyping
- Advantages
 - Emphasis on planning and analysis which forces analysts to spend a significant amount of time on analysis, thereby understanding the problem thoroughly before designing and coding the system.
 - User requirements are determined and documented in detail before embarking on the risk of design and implementation
- Limitations
 - Users might not be able to articulate requirements accurately
 - User requirements might change as time passes, and is usually difficult to revisit early steps in the processes as necessitated by the changes.

The Structured Approach

System development using structured programming, structure analysis, and structured design techniques. Sometime referred to as structure analysis and design techniques (SADT).

Structured Programming

- Structured Program
 - A program or program module that has one beginning and one ending, and each step in the program execution consists of sequence, selection, or repetition constructs.
- Top-down Programming (Modular Programming)
 - Divides complex programs into a hierarchy of program modules, the top module calls the low level modules to accomplish the task.
- Structured Design
 - A technique providing guidelines for deciding what the set of programs should be, what each program should accomplish, and how the programs should be organized into a hierarchy
 - Two main principles
 - * Modules should be designed so that they are (1) Loosely coupled: each module is as independent of other modules as possible. Later modification will not affect other modules. (2) Highly cohesive: each module accomplishes one clear task. Easy to understand, change of a module will not affect other modules.
 - Structured design's quality is defined in terms of how easily the design can be understood and modified later.
 - Later developments of structured design
 - * File and database design techniques to be used with Structured Design
 - * User interface design techniques to be used with Structured Design
- Structured Analysis
 - A technique that helps the developer define what the system needs to do (the processing requirements), what data the system needs to store and use (data requirements), what inputs and outputs are needed, and how the functions work together overall to accomplish tasks.

- Principle: focus on requirements, don't commit to specific technologies.
- Models used in structured analysis
 - * Data flow diagram (DFD): A graphical model showing the inputs, processing, storage, outputs of a system produced in structured analysis.
 - * Entity-relationship diagram (ERD): A graphical model of the data needed by a system, including things about which information is stored and the relationships among them, produced in structured analysis and information engineering.

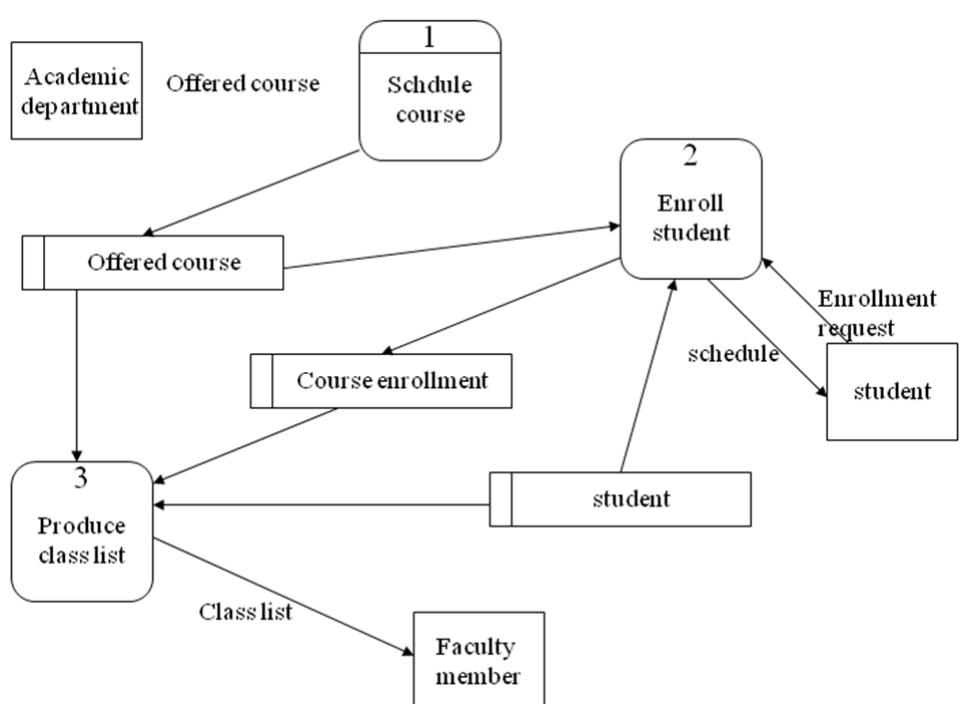


Figure 1.2: Sample DFD

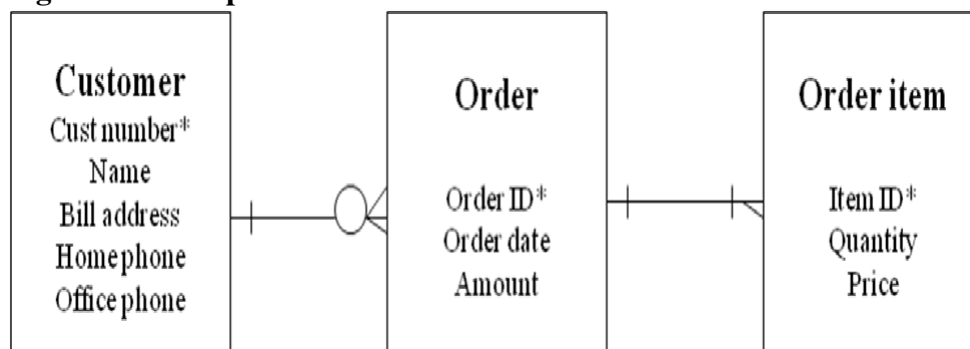


Figure 1.3: Sample ERD

Weakness of the Structured Approach

1. Address some, but not all, of the activities of analysis and design.
2. Techniques are not comprehensive and rigorous enough to make system development more like engineering discipline and less like an art.
3. Transition from Data flow diagram to structure chart is difficult.
4. Focus on process but data is more important (although SADT include data modeling and database design)
5. Need an overall strategic system plan

The Information Engineering (IE) Approach

A system development methodology that focus on strategic planning, data modeling, and automated tools. Thought to be more rigorous and complete than the structure approach.

Differences between SADT and the information engineering approach(IE):

1. IE begins with overall strategic planning
2. SADT focus on processes, IE focus on data
3. IE has a more complete life cycle and the use of CASE tools.
4. IE is more rigorous, but less flexible
5. Now Fewer companies use the IE approach exclusively
6. SADT's and IE's concepts are merged and be referred to as the traditional approach.

The Object-Oriented Approach

An approach to systems development radically different from the traditional approach, which views an information system as a collection of interacting objects that work together to accomplish tasks. Corresponding to Structured approach,

object-oriented approach consists of object-oriented analysis, object-oriented design, and object-oriented programming.

- Object
 - a thing in the computer system that can respond to messages.
- Object-oriented Analysis (OOA)
 - A technique that defines all of the types of objects that do the work in the system and shows how the objects interact to complete tasks.
- Object-oriented Design (OOD)
 - A technique that defines all of the types of objects necessary to communicate with people and devices in the system and refines the definition of each type of object so it can be implemented with specific language or environment
- Object-oriented Programming (OOP)
 - Writing statements in a programming language to define what each type of object does, including the messages the objects send to each other.

1.1.3. Object Oriented Analysis and Design

Welcome to the world of object-oriented analysis and design. You are about to embark on learning a method which is fun, elegant, easy to understand, powerful and comprehensive.

The reasons for using object oriented design phase instead of going directly to implementation include:

1. The design phase deals with a higher level of abstraction than the analysis phase. Hence the system in is a finer detail form, enhancing its development.
2. The design phase exposes flaws that would not have otherwise been notices in the analysis phase.

3. Design techniques such as information hiding, abstraction, modularity, polymorphism etc, enhance the system's performance and development. These techniques are not much implemented in the analysis phase, if implemented at all.

Object modeling is useful for designing computer systems, whether those systems are to be implemented in object-oriented languages or not. Most designs are likely to need more than an object-oriented language, such as a database. Object modeling also has a use outside of the design of computer systems. It is an excellent analysis method, and it can be used in business process re-engineering, in social science research, or any complex environment where it is important to capture the structure and functionality of some world.

Object-oriented programming is based on the principle of recursive design.

1. Everything is an object

Actions in OOP are performed by agents, called instances or objects. Each agent has a part to play, and the result is produced when all work together in the solution of a problem.

1. Objects perform computation by making requests of each other through the passing of messages

Actions in OOP are produced in response to requests for actions, called messages. An instance may accept a message, and in return will perform an action and return a value.

Information Hiding

Notice that I, as a user of a service being provided by an object, need only know the name of the messages that the object will accept. I need not have any idea how the actions performed in response to my request will be carried out. Having accepted a message, an object is responsible for carrying it out.

Elements of OOP - Receivers

Messages differ from traditional function calls in two very important respects:

1. In a message there is a designated receiver that accepts the message

2. The interpretation of the message may be different, depending upon the receiver

Behavior and Interpretation

Although different objects may accept the same message, the actions (behavior) the object will perform will likely be different. The determination of what behavior to perform may be made at run-time, a form of late binding. The fact that the same name can mean two entirely different operations is one form of polymorphism, a topic we will discuss at length in subsequent chapters.

1. Every object has its own memory, which consists of other objects.

Each object is like a miniature computer itself - a specialized processor performing a specific task.

Non-interference

It is important that objects be allowed to perform their task however they see fit, without unnecessary interactions or interference with other objects.

1. Every object is an instance of a class. A class groups similar objects. The class is the repository for behavior associated with an object.

Behavior is associated with classes, not with individual instances. All objects that are instances of a class use the same method in response to similar messages.

Hierarchies of Categories

At each level of abstraction I have certain information recorded. That information is applicable to all lower (more specialized) levels.

1. Classes are organized into singly-rooted tree structure, called an inheritance hierarchy.

Information (data and/or behavior) I associate with one level of abstraction in a class hierarchy is automatically applicable to lower levels of the hierarchy.

- Inheritance promotes Re-use: When new objects are created which are similar to other objects, they can have many of their attributes and operations ready defined.

- Inheritance promotes clarity: The notion of breaking the world down into hierarchies of types is not as old as the hills, but it goes back at least as far as the Ancient Greeks: all men are mortal, Socrates is a man, so Socrates is mortal. The reductionist, classification approach is embedded in Western thought, and should be natural for most people to follow.

Subclasses can alter or override information inherited from parent classes:

- All mammals give birth to live young
- A platypus is an egg-laying mammal

Inheritance combined with overriding are where most of the power of OO originates.

1.1.4. Computing as Simulation

The OOP view of computation is similar to creating a universe of interacting computing objects

- Similar to the way in which a committee or club might be organized
- Also very similar to a style of simulation called discrete event-driven simulation
- Easily underestimated advantage of this view – power of metaphor.

Metaphor and Problem Solving

Because the OOP view is similar to the way in which people go about solving problems in real life (finding another agent to do the real work!), intuition, ideas, and understanding from everyday experience can be brought to bear on computing. On the other hand, common sense was seldom useful when computers were viewed in the process-state model, since few people solve their everyday problems using pigeon-holes.

So why are Object Oriented concepts important?

1. Objects provide for modularity and encapsulation. Modularity implies that you can create units that can be used throughout a system or organization. Once an object is defined, this object definition and its properties can be used by other analysts and developers throughout the organization. Encapsulation or the ability to hide information means objects have public and private information, implying you can change the private information without affecting the other objects that use the public information.
2. Classes support the concept of reusability. Through inheritance, you have the basic information necessary to create additional objects without having to define the base information again.
3. Abstraction means objects are analyzed until they are uniquely identified. Through these multiple analyzation iterations, you establish classes and subclasses. This iteration process supports the analytical process and helps to ensure a thorough analysis of the system.
4. Persistence means the classes and objects are permanent and will be available for use throughout the life of the system.

1.1.5. Criticisms of Object Oriented Analysis and Design

1. Lack of standardization. This criticism has been taken care of by the adoption of the Unified Modeling Language (UML) by the Object Management Group (OMG) in November 1997. UML was developed by three of the leading proponents of object-oriented development, Grady Booch, James Rumbaugh and Ivar Jacobson, and provides a language for modelling systems. The OMG is an organization whose mission is to introduce standardization to object software; therefore, this adoption represented standardization in the analysis process.
2. The costly nature of adopting this paradigm. Change to OOAD requires the infamous paradigm shift, meaning you have to completely change your way of thinking and change your business processes as well as invest in training in order to ensure the staff is ready to accommodate the changes. This requires an investment in not only money but time.

3. Problems associated with the concept of reusability. It also must be kept in mind that software developers who have used traditional methods for a long period of time tend to think in terms of process and, therefore, may find it difficult to change. The investment, however, may return dividends in the long run. A study comparing the data flow diagram method to the object-oriented analysis method showed that there was a longer learning curve associated with object-oriented analysis but, once learned, the object-oriented analysts performed better than the data flow diagram subjects in analyzing a system.
4. The applicability of adopting it for legacy systems. Finally, there is some question as the appropriateness of applying object-oriented techniques to legacy software. Object-oriented methodology starts with the object and this object drives the definition of further needs. With legacy system, the data and its associated model already exist. You also have analysts steeped in traditional methodology more comfortable building the data model first. The good news is that once an analyst uses an object-oriented approach he is resistant to going back to traditional methods.

1.1.6. Benefits of Object-oriented Approach

1. Naturalness: People tend to think about the world in terms of tangible objects
2. Reuse: Classes can be used again and again whenever they are needed. There is less need to “reinvent the wheel”

1.2. Summary

1. Object-oriented programming is not simply features added to a programming language. Rather, it is a new way of thinking.
2. Object-oriented programming views a program as a community of agents, termed objects. Each object is responsible for a specific task.
3. An object is an encapsulation of state (data values) and behavior (operations).
4. The behavior of objects is dictated by the object class. • An object will exhibit its behavior by invoking a method (similar to executing a procedure) in response to a message.

5. Objects and classes extend the concept of abstract data types by adding the notion of inheritance.

Learning Activities

1. Describe two of the following objects in terms of their attributes and operations: motor car, sheep, kite, hospital, elephant, garden, school, bacon, teddy bear, bank customer, bus.
2. Devise a simple inheritance hierarchy for two of the following: geometry, sports teams, politicians, rodents, viral infections, restaurants.

Revision Questions

Example  Briefly explain the concept of an object and hence describe three characteristics of an object.

Solution: for revision



EXERCISE 1.  Briefly explain the UML

EXERCISE 2.  Discuss Object Oriented Principles and Concepts

References and Additional Reading Materials

1. Grady Booch (2007). Object oriented analysis and Design with applications, ISBN-13: 978-0201895513
2. J W. Satzingr, Robert B.Jackson, Stephen D. Burd (2004). Object oriented analysis and Design with Unified Process, ISBN-13: 978-0619216436

LESSON 2

Information System Development

Learning outcomes

By the end of this topic you should be able to;

1. Outline the problems encountered in information system development both from a user perspective, client perspective and developer perspective.
2. Examine why the above problems are encountered, and how they can be avoided.
3. Discuss the different project life cycles in system development.
4. Carry out extensive case studies of real life systems in their failure or success in development.

2.0.1. Problems in Information Systems Development

Most software developers go straight into coding without first understanding the issues surrounding the problem or consulting the right people who are involved in the problem. Analysis is a way of understanding what needs to be done before we begin trying to do it, and design is a way of checking that the planned action or solution really meets the needs of the situation. An understanding of potential problems is an essential precursor to systems analysis and design.

The Problems

1. End user perspective:
 - (a) What system? I haven't seen a new system: Vapourware describes a software product that is much talked about, but never released to its intended users. An astounding proportion of information systems development projects fail to deliver any product at the end.
 - (b) It might work but its dreadful to use!: This relates to systems that are unpleasant or difficult to use. Systems might fail to meet the criterion of usability in a number of ways, including: poor interface design, inappropriate or illogical sequence of data entry, incomprehensible error messages, unhelpful 'help', poor response time and unreliability in operation. The table below illustrates examples:

Systems Characteristic	Example
Poor interface design	A web page with yellow text on a white background
Inappropriate data entry	A system where the backspace key sometimes deletes whole words
Incomprehensible error message	A system message that says 'error # 13452'
Unhelpful 'help'	A system message that says 'wrong date format –try again'
Poor response time	
Unreliability in operation	

Figure 2.1 Information Systems Development Problems

1. It's very pretty- but does it do anything useful?: A system may appear well designed and easy to use, but still not do the 'right' things.
2. A client's perspective
 - (a) 'If I had known the real price, I' never have agreed': it is almost routine in many organizations for information systems to exceed their budget. Any project that overruns its budget can reach a point where the total costs outweigh the benefits that will be provided on completion.
 - (b) 'It's no use delivering it now': A project that is completed late may no longer be of any use. Many other kinds of projects are time-critical.
 - (c) A messy installation: Once a new system gets a bad press it can be very difficult to overcome the resistance of the people who are expected to use it, particularly if there is an existing alternative available to them.
 - (d) 'I didn't want it in the first place': Organizations are complex and political by nature. There are sometimes conflicting ideals and ambition and the play of power within an organization.
 - (e) Changing needs: Sometimes by the time the system is completed. The requirements are no longer the same as they were thought to be at the beginning

Requirements can change for many reasons:

- Project timescales can be very long, and the business needs may change in the meantime.
- Users naturally tend to ask for more, as they learn more about what is available.
- External events can have a dramatic impact – for example a currency crisis that led to the collapse of a significant market might render a new information system pointless.

1. A developer's perspective

- (a) We built what they said they wanted: Changes to the requirements can be difficult for a developer.
- (b) There wasn't enough time to do it any better: In every project, there are pressures from outside that limit the ability of the development team to achieve excellence.
- (c) Don't blame me – I have never done object-oriented analysis before: In a successful information system development team, the members must possess a harmonious blend of skills that are appropriate to the needs of the project.
- (d) How can I fix this, I don't know how it's supposed to work: This complaint is often heard from programmers who have been asked to modify an existing program, and who have then discovered that there is no explanation of what it does or how it works.
- (e) We said it was impossible, but no one listened: Just like client managers, system developer can sometimes be overwhelmed by organization politics. At times this means that the project is forced on an unwilling team who don't believe that it is technically feasible to achieve the project's goals.
- (f) The system's fine, the users are the problem: A few information systems professionals, usually those who understand least about business and organizations are sometimes prone to blame the user for everything.

2.0.2. Why Things Go Wrong

Quality Problems

The quality of a product is in terms of its ‘fitness for purpose’. It is therefore necessary to know for what purpose the system is intended and how to measure its fitness. Both parts of this can be problematic at times.

- The wrong problem A primary cause of system failures in general is that some projects are started with no clear idea about exactly what are the nature and goals of the client organization.
- Neglect of the content This can take the form of a system that is too difficult to use, since the designers have taken insufficient account of the environment in which its users work, or the way that they like to work.
- Incorrect requirements analysis Even if the aims are clear at the onset, many pitfalls arise particularly if the development team doesn’t not have the right skills, the right resources or enough time to do a good job. However, even if none of these present a difficulty, the project can still fail if the techniques being applied by the team are inappropriate to the project.
- Project carried out for the wrong reason

Productivity Problems

Productivity relates to the rate of progress of the project, and the resources that it consumes along the way.

- Requirements drift This means that the users’ requests change over time.
- External events Depending on the environment in which the organization operates, decisive external events can be impossible to anticipate. For example, a project to build a distributed information system that is to operate on new state-of-the-art computers communicating over public telephone circuits may be sensitive to external factors such as the reliability of the telephone network and call pricing.
- Poor project management d. Implementation not feasible Some projects are overambitious in their technical aims, and this may not become evident until an attempt is made to implement the system.

2.0.3. Avoiding The Problems

There are two major areas of focus if we intend to produce information systems within budget, on time and providing the required functionality: Project Life Cycles and Management of Information systems.

2.0.4. Project Life Cycles

Subdividing the process of software development produces what is known as a life cycle model.

- **Traditional Life Cycle**

This model is also known as the waterfall life cycle model (You should have learnt this in software engineering – please research further). The traditional life cycle has been used for years but is subject to several criticisms:

- Real project rarely follow such a simple sequential life cycle. Project phases overlap and activities may have to be repeated.
- Iterations are almost inevitable, because inadequacies in the requirements analysis may become more evident during design, construction and testing.
- A great deal of time may elapse between the initial systems engineering and the final installation.
- The model tends to be unresponsive to changes in client requirements or technology during the project.

- **Prototyping**

A prototype is a partially complete system that is built quickly to explore some aspect of the system requirements that is not intended as the final working system. It is differentiated from the final production system by some initial incompleteness and perhaps a less resilient construction. A prototype may be constructed with various objectives in mind:

- Used to investigate user requirements.
- To investigate the most suitable form of interface.

- To determine whether a particular implementation platform can support certain processing requirements.
- To determine the efficacy of a particular language, a database management system or a communications infrastructure.

Prototyping has the following advantages:

1. Early demonstration of a system functionality helps in identifying any misunderstandings between the developer and the client.
2. Client requirements that have been missed are identified.
3. Difficulties in the interface can be identified.
4. The feasibility and usefulness of the system can be tested, even though, by its very nature, a prototype is incomplete.

Prototypes disadvantages:

1. The client may perceive the prototype as part of the final system, may not understand the effort that will be required to produce a working production system and expect it delivery soon.
2. The prototype may divert the attention from functional to solely interface issues.
3. Prototyping required significant user involvement.
4. Managing the prototyping life cycle required careful decision making.

● **Incremental Development**

One way to overcome the problems of the traditional model is to view successful large systems as starting out as successful small systems that grow incrementally.

● **Unified Software Development Process**

This process reflects the current emphasis on iterative and incremental life cycles. This incorporated the UML and comprises four phases:

1. Inception: Determining the scope and purpose of the project.

2. Elaboration: Requirements capture and determining the structure of the system.
3. Construction: to build the software system.
4. Transition: product installation and roll out.

2.0.5. Managing Information Systems Development

1. User involvement A key factor in maximizing the chances of success is ensuring that there is continued and effective user involvement throughout the project.
2. Methodological approaches One of the major influences on the quality of systems developed is the software development approach adopted. If the approach used is not appropriate for a particular type of application, then it may limit the quality of the system being produced.
3. CASE (Computer Aided Software Engineering Tools). These provide support for many of the tasks the software developer must perform.

CASE STUDY: The London Ambulance Service Computer Aided Despatch System (LASCAD)

LASCAD is being examined because it is an example of a requirements problem. In 1992, the London Ambulance Service's (LAS) entire system was managed from a central location at the LAS headquarters in Waterloo. On average between 2000 and 2500 calls were received daily, 60% of which were requests for emergency services. The LAS devoted 55% of its staff and 76% of its budget specifically to emergency response. The original design of the LASCAD system was first proposed in 1987. It was modified in 1989 and then, due to gross overspending by about 300%, the project was cancelled in 1990. However, the national mandate to reduce emergency response time pushed the LAS to look into a computerized system once again (Dalcher, 1999).

According to Wikipedia, LAS remains the 'largest ambulance service in the world that does not directly charge its patients for its services'. Before 1992, The LAS emergency system was run completely manually and consisted mainly of the following issues and tasks:

1. The call taker receiving large number of calls and then records all the status details on a paper form. He then locates the incident on a map and puts the form on a conveyor belt.
2. Control staffs select the next job from the forms on the conveyor belt then identify a regional allocator.
3. A resource allocator analyzes the locations and status of the ambulances and allocates the call to an available unit and records the assignment on the form.
4. A dispatcher informs the ambulance to which the call was allocated and provided the details of the call.

Proposed Improved System

The government-imposed stipulations that calls be responded to within three minutes, in addition to the obvious deficiencies of the manual dispatch system, this led those in charge of LAS seek out a computer-based alternative. LAS management decided to create a new one and proceeded to gather requirements without receiving input from ambulance crews or dispatchers (Dalcher, 1999).

Rather than simply assisting dispatchers, this completely computerized system would do nearly everything automatically. The simple sequence of steps performed by a person would be to answer the phone, enter incident data into a computer terminal, and then respond if the system displayed exception messages resulting from no ambulances being available for longer than 11 minutes. The software would map the location of calls. The system would then use this map data and the location status details provided by an automatic vehicle locating system (AVLS) to find and dispatch the available ambulance closest to the incident's location. This system would be implemented not incrementally, but all at once (Dalcher, 1999).

The Problems

As part of the attempt to save money, the LAS decided to reuse some of the hardware that had already been purchased when working on the failed project instead of purchasing hardware that was either more up-to-date or more suitable for the new system. All companies that submitted bids for the project, greater than £1.5 million were immediately thrown out. This was an unreasonably low price, especially

considering the fact that even after £7.5 million had been poured into the previous attempt at a CAD system, the project had failed (Dalcher, 1999). A further hard constraint was placed that the project be completed in 11 months. Any bids not meeting this constraint were not considered.

The problems faced included:

1. The AVLS was unable to keep track of the ambulances and their statuses in the system. It began sending multiple units to some locations and no units to other locations.
2. The system began generating a great quantity of exception messages on the dispatchers' terminals such that calls got lost.
3. As more and more incidents were entered into the system, it became increasingly clogged.
4. The software system did not function well when given invalid or incomplete data regarding the positions and statuses of ambulances.
5. A memory leak in a small portion of the code caused memory that held incident information on the file server, to be retained even after it was no longer needed. Eventually when the memory filled up, it caused the system to fail.

The LAS team – not a software development team - went through the process of gathering requirements and specification, without consulting ambulance operators, dispatchers and other key users of the system. The requirements document was 'highly detailed and extremely prescriptive', very much a direct contradiction of the requirements process' focus on the 'what' as opposed to the 'how' (Dalche, 1999). The software development process was flawed. No quality assurance was performed, configuration management was absent, agreed-upon changes were not tracked and test plans were not written. The LAS also failed because of the inappropriate algorithms chosen for scheduling and despatch. They utilised an algorithm that sent the nearest ambulance first, leading to ambulances getting farther and farther out of their 'home' regions.

The Turn-around

While it is true that the CAD system's software errors demonstrate 'carelessness and lack of quality assurance of program code changes,' had the LAS paid more

attention to selection process of the developing organization and imposed more reasonable and realistic expectations, the CAD system likely would not have failed as it did (Page et al , 1993). In 1996, a new Management of LAS adopted a new approach to the CAD system. A participative approach utilizing prototyping was adopted to help involve the users and instil ownership and acceptance of the system. A very slow and deliberate approach was adopted and provided time for participation and iteration. A great deal of attention was paid to testing and training and the system was only implemented when it was felt to be ready, not just on a technical sense but when the users were convinced about the its capabilities (Fitzgerald and Russo, 2005).

In relation to technology, a new hardware platform was chosen, as the old system was essentially a PC architecture, which was not thought to be adequate for a command and control system (Fitzgerald and Russo, 2005). The new system went live on 17th January 1996. The initial system was a very basic system enabling the operators to receive a call and enter the details of an incident directly into the system. The computerized Gazetteer, using a Postal Address File, looked up the location and provided a map reference within one second. The system provided the controller with information on which of the 150 or so deployment points around London was the nearest to the incident and the controller would then dispatch an appropriate ambulance. The system additionally provided the location of the nearest hospital for the controller to pass to the ambulance crew. The new CAD system was implemented with few problems and it provided immediate benefits (Fitzgerald and Russo, 2005). A significant enhancement was introduced in September 1996, when 'early call viewing' was introduced. Once the call-takers had established the address of the incident that information was immediately made available to the controllers to begin the dispatch process, that is, before the call had finished. The result was that the annual performance rates improved significantly (Fitzgerald and Russo, 2005).

Learning Activities

We shall use a class timetabling system as our case study for this unit. The new class timetabling system is aimed at minimizing timetable class clashing while maximizing the resources of the university in a minimal amount of time. The system should be implemented to run on the university LAN and each JKUAT member should use their login ID and password to access the system; in case the user login details are wrong, the system should indicate to the user an error message. The system should be online 24 hours a day. A user should also be able to print or save the timetable to an external disk such as a flash drive. The system should be able to indicate an error message if the specified printer or disk is not found. The actions that the users can perform include:

1. Querying the times that a class is.
2. Querying the room that a class is in.
3. Free times at a particular day or period.
4. Free rooms at a particular day or period.
5. Query the number of classes taking place at a particular time.

Any of these queries should return either a positive or a negative response, for example, there may be no free rooms for the period of time the user specified. The class timetabling system should be updated only by the timetable master. Other users should not be able to make any changes to the timetable. The timetabling master can update the timetable by inputting new data or modifying existing data. The system should display an alert if data has been input or modified but no update has been called to be done on the timetable. While updating or modifying the timetabling, there might be some conflicting data required for the timetable to be scheduled properly, for example, when two instances of the room Lab6 have been entered. The system should be able to display an alert when this happens.

Revision Questions

Example . Explain the main stages of systems development from an object-oriented point of view.

Solution: for revision



EXERCISE 3.  Explain the significance of each of the following views in the development of a system: a) Use case view b) Analysis view c) Design view

EXERCISE 4.  Describe the principal characteristics of an object-oriented software system

References and Additional Reading Materials

1. Grady Booch, James Rumbaugh, Ivar Jacobson (2005). The Unified Modeling Language User Guide, ISBN-13: 978-0321267979
2. Richard Lee , William M.Tepfenhart (2000). UML and C++ - A practical guide to object oriented development” , ISBN-13: 978-0130290403

LESSON 3

Identifying and Defining Objects and Classes

Learning outcomes

Upon completing this topic, you should be able to:

1. Differentiate between Objects and Classes.
2. Identify Objects and Classes from a problem Definition.
3. Translate Classes into database tables.
4. Outline features of generalization, and construct generalization and aggregation models.
5. Examine the different types of object and class relationships.

3.1. Objects and Classes

Coad and Yourdon defined an object as an abstraction of something in a problem domain, reflecting the capabilities of the system to keep information about it, interact with it, or both, for example a Person, A shirt etc. The UML specification gives a definition that illustrates the strength of the connection between the three concepts of an object, instance and class. An object represents a particular instance of a class. Objects that are sufficiently similar to each other are said to belong to the same class. Instance is another word for a single object. The idea that instances belong to a class logically implies that there must be a test that determines to which class an instance belongs.

There are two distinct types of logical similarity which must be tested:

1. All objects in a class share a common set of descriptive characteristics.
2. All objects in a class share a common set of valid behaviors.

For example, if we are interested in developing an Academic Records Information System (ARIS), the table below defines a Student Class template.and each Student object fills in its own unique attribute values.

Attribute Name	Data Type	Value
Name	String	Reagan Okeyo
StudentId	String	123457
Birthdate	Date	January 1st, 1987
Address	String	123 Tom Mboya Street
Major	String	Information Technology
Gpa	Float	3.5
Advisor	???	To be determined
CourseLoad	???	To be determined

Figure 3.1: Student Class Template

Because most systems need to store information in a database, we need to know how to convert a class into a database. The above tabular definition of a class may need to be stored in a database. Classes to be stored in a database can be mapped onto tables in the following ways:

1. One class is considered as one table.

2. The class' attributes are considered as the columns. If the class has more than one attribute, then additional columns are involved.
3. The instances of the classes are the rows.
4. In case of an object that has cardinality greater than one, and then a new table is formed.

3.2. Analysis - The Rudiments of an Approach

The first reaction to any project with any complexity is "how do we get started?". Where do you begin? The starting point for object-oriented analysis is to identify candidate objects and their relationships. In fact, this is the be-all and end-all of object-oriented analysis. All we are going to do in the rest of the course is give you some tools to look for objects. But for the present, we will rely on native wit. Begin by looking at the real world. Open your eyes and your ears. The first stage can be a simple brain-storming of the possible objects. One method is to go through all the nouns in any documentation about the world you are analyzing, and considering these as candidate objects. Let us suppose that you are analysing a system for a motor museum. The system is to produce a multimedia guide to the museum. A preliminary list of objects might be: car, bus, vehicle, number plate, exhibit, manufacturer, date of manufacture, value, position, weight, size, photograph, tools, shop, garage, ticket, owner, history No doubt we could go on. However, the above list is probably enough to illustrate our needs.

3.2.1. Removing Synonyms and/or Generalizing

The first stage is to try and group together items which are either just different names for the same thing, or whether there is a common generalization. The obvious thing above is to examine the following group car, bus, vehicle, number plate, exhibit, photograph, tools, garage It would seem that these could all be grouped together under the object exhibit. The question is, can these all be considered to be one object. In this case, the answer is probably yes. In the museum the behavior of cars, photographs, and garages are not radically different, so we can group them together. We can differentiate between the individual objects on the basis of an attribute, say type. We are then left with the list of candidate objects: exhibit, man-

ufacturer, date of manufacture, value, position, weight, size, shop, ticket, owner, history.

3.2.2. Look for Attributes

Attributes are themselves objects, but with trivial behavior - essentially all you do is record their value. Considering the above list, the following are probably attributes of exhibit: exhibit, date of manufacture, value, position, weight, size, owner So we have the exhibit object as: And we are left with the candidate objects: exhibit, manufacturer, shop, ticket, history We might have considered manufacturer as an attribute, but it may be useful to separate out manufacturer information later in the design. So we keep it for the time-being. The time to fold it in as an attribute is when we find it has no discernible behavior of its own.

3.2.3. Irrelevant Objects

Sometimes an object is irrelevant because of what you are designing the system for. Here the ticket object may be very relevant for a sales desk system, but not for a multimedia system. Likewise, the shop is irrelevant. So we strike these out and we are left with the following objects. exhibit, manufacturer, history We have these objects potentially linked in the following way.

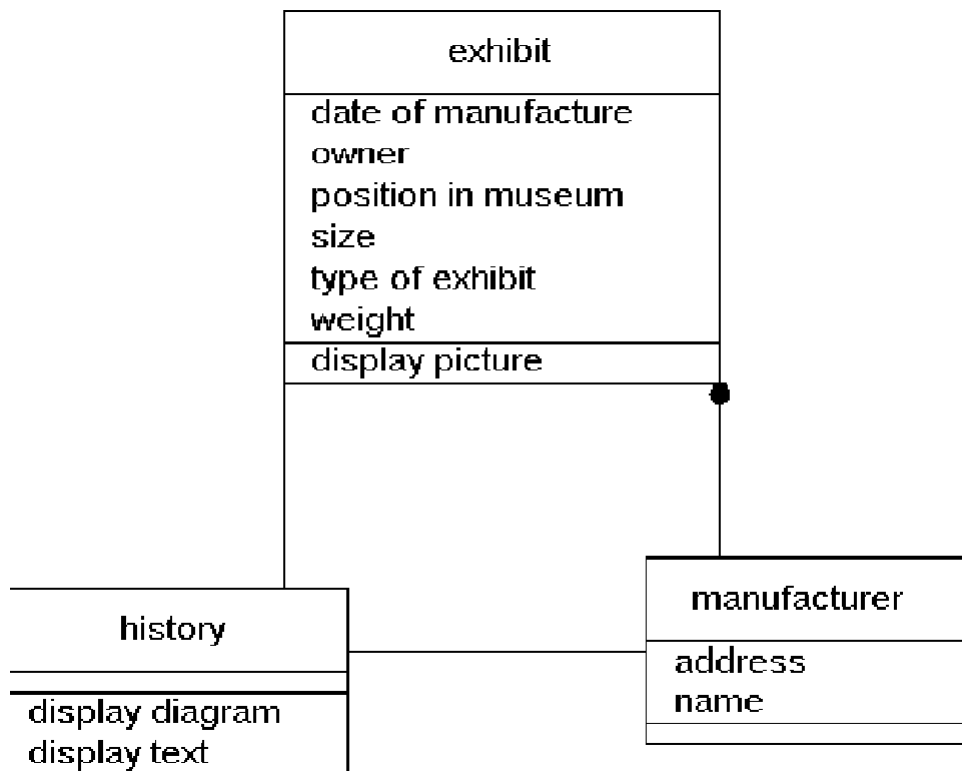


Figure 3.2: Exhibit Initial Class Diagram

Both exhibits and manufacturers may have histories. By factoring out manufacturers, the system may be able to trace from an exhibit not only its history, but also that of its manufacturer, and maybe allow someone to search for all the objects in the museum held by a given manufacturer.

3.3. The Process of Development

The approach to development recommended here is an iterative one. It involves repeated refinement of the object model. The process needs to be controlled by an appropriate project management process, involving reviews and check-pointing.

The approach to development recommended here is an iterative one. It involves repeated refinement of the object model. The process needs to be controlled by an appropriate project management process, involving reviews and checkpointing.

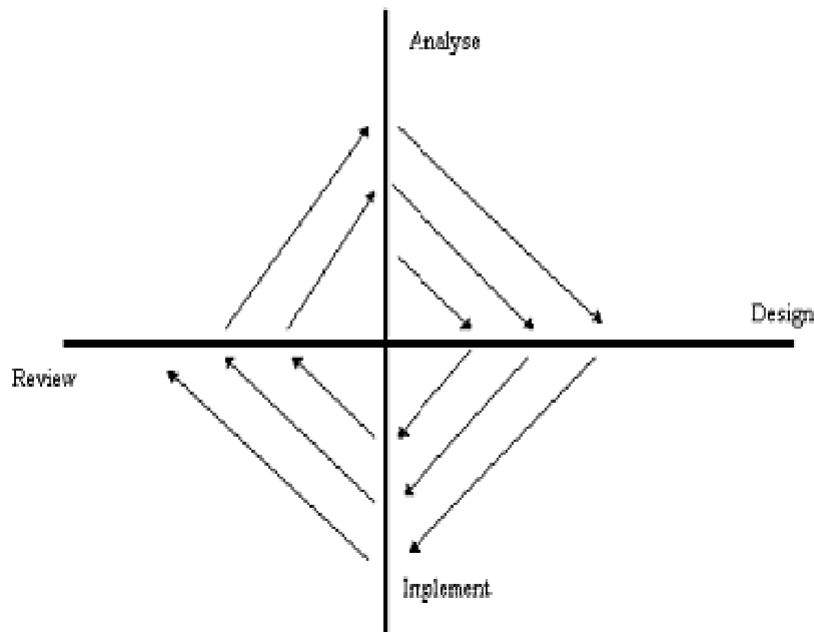


Figure 3.3: Process of Development

There are a variety of techniques for driving this cycle:

Prototyping

The design of the process progresses by the construction of a series of prototypes. The prototypes are constructed using the most productive tools in terms of speed of development. The final system, when constructed, is functionally equivalent to the final prototype, but it is implemented in the most suitable technology for use of the system. For example, a patient monitoring system, requiring lots of graphical displays from a digital feed, might be prototyped in a spreadsheet because the spreadsheet has very powerful graphics. However, when the system is built it would have to be implemented to run on a specific computer with a limited display and limited amount of memory, without a spreadsheet license.

Using iterative design methods, a series of prototypes could be developed using a spreadsheet until functionality is finally agreed. The final prototype would then be used as an active specification for the final system which might be written in C.

Prototyping is useful in the following contexts:

- at the beginning of a project
- in research-oriented projects where there needs to be some exploration of different ways of solving problems
- where there is a need to clearly define interfaces
- where the client needs reassurance that progress is being made

The weaknesses of prototyping are:

- the prototype is often taken on as a final system
- there can be difficulty explaining a long lag between first prototype and final system
- inevitably there will be some differences between the final system and the final prototype because of different interface mechanisms.

Evolutionary Development

This process involves a staged implementation of the system. Analysis and design of the next stage is after successful implementation of the previous stage. Evolutionary development is useful if:

- a partial implementation of the proposed system is of some use
- it is not replacing another system which cannot be integrated appropriately at each stage of development

The weaknesses of evolutionary development are:

- disruption caused by each stage of implementation
- difficulty controlling the end-point of development

Boxing

Computer systems development is constrained by money, time and personnel available to develop, implement and install. The idea of boxing is to drive the iterative process of development by check pointing at key stages according to one of the three constraints. Time-boxing involves setting a time limit on each iteration of the

cycle. As the time limit nears, the system is implemented, irrespective of whether the planned functionality is fully implemented. The system is reviewed, and the plans for the next stage take into account what has been achieved and what has been learnt. Money-boxing is similar to time-boxing, except that the reviews are triggered by expenditure limits. Personnel-boxing involves controlling the process using the time of certain personnel. Boxing has an advantage of driving the system, and forcing changes of plan as the system development progresses.

Charters

Rather than produce a contract defining all functionality to be delivered in a cycle, it is possible instead to construct a charter. A charter prioritizes the functionality which should be delivered in the cycle. The charter can over-specify what is expected in the cycle, in case progress is faster than anticipated.

Why Object Modeling Lends Itself to Iterative Methods

The problem with traditional systems developments has been that notations differ in each of the phases of development. There is then a translation problem and consistency problem at each phase transition. Consequently changes upstream have always been resisted. Object modeling uses the same notation for both analysis and design, and if object-oriented languages and databases are used, the notation persists through to implementation. Design is essentially a fleshing out of the analysis, by further refining and adding detail to the objects produced by the analysis, and by adding new objects, say for the purposes of providing interfaces. Object modeling, by using a consistent notation in each phase, results in more productive CASE tools which can generate the code outlines for the implementation.

Generalization

In the UML specification (OMG, 2001) generalization is formally defined as: a taxonomic relationship between a more general element and a more specific element. The more specific element is fully consistent with the more general element and may contain additional information. The main application for generalization in object-orientation is to describe relationship of similarity between classes. A second benefit comes from the relative ease with which a hierarchy can be extended to fit a changing picture.

Other features of generalization not mentioned in the definition are:

- Inheritance: The mechanism of implementing generalization and specialization in an object-oriented programming language.
- Transitive operation: The relationship between two elements at adjacent levels of hierarchy 'carries over' to all more specialized levels.
- Disjoint nature of generalization hierarchies: an object cannot belong to two classes, for example, a cat cannot be both a mammal and a reptile.

Message Passing

In an object oriented systems, objects communicate with each other by sending messages. Message passing allows objects to encapsulate their internal details from other parts of the system, thus minimizing the knock-on effect of any changes to the design or implementation.

Aggregation

We can make up objects out of other objects. This is known as aggregation. The behavior of the bigger object is defined by the behavior of its component parts, separately and in. Here is a simple example of a juggler:

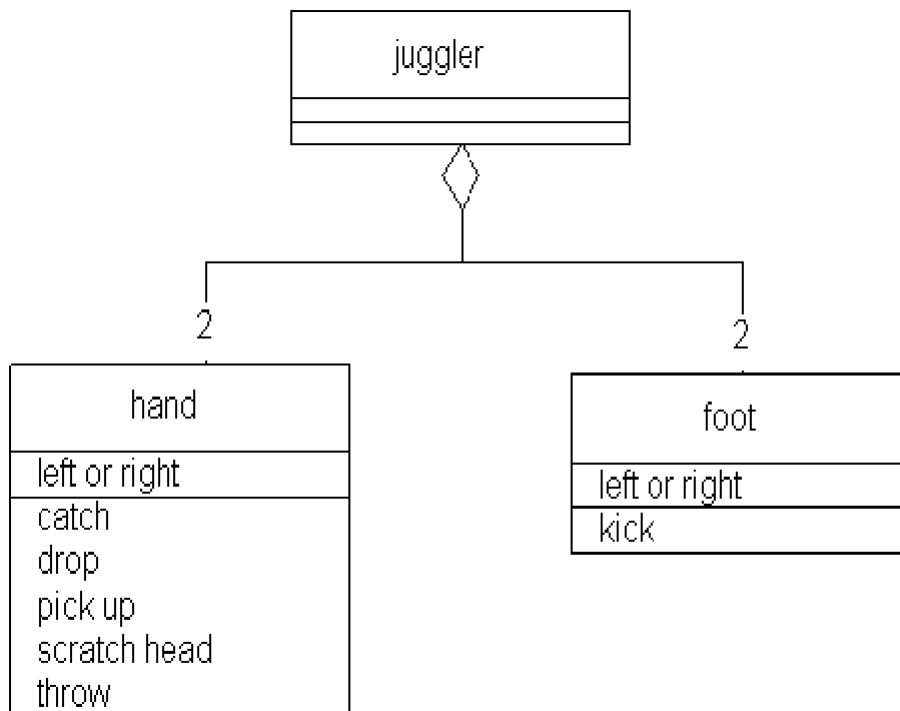


Figure 3.4: Aggregation Example

A juggler has two hands and two feet. He or she uses hands to catch, drop, pick up and throw a ball, and perhaps from time to time scratch his or her head. He or she may also kick a ball with his or her foot. By analyzing our juggler object and breaking it down into component objects, we now have a better understanding of our object. Also, we have discovered two new operations, scratch head and kick. The diamond in the diagram indicates that one object is made up of another object. The numbers are indicative of how many - more will be said about this shortly. Now the behavior of a juggler is entirely defined by the behavior of his or her hands and feet. Of course, real jugglers are made of quite a bit more, but for the purposes of considering their juggling skills we can focus on just the bits of them that are involved in juggling. Hands and feet could be broken down into their constituent parts, say palms and fingers, soles and toes. However, that does not seem to help us to understand juggling, so the decomposition above is probably enough.

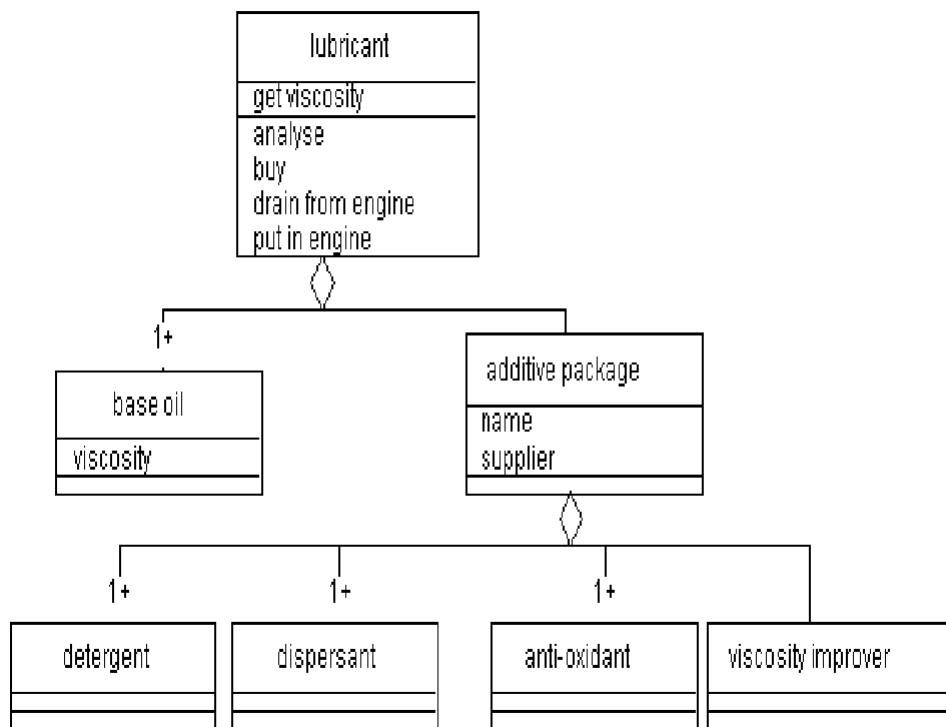


Figure 3.5: Another Aggregation Example

A car engine lubricant is made up of a number of base oils blended with an additive package. The additive package consists of one or more detergents to keep engine surfaces clean, one or more dispersants to suspend particles in the oil to be carried to the filter, one or more anti-oxidants to slow up the thermal decay of the oil, and a viscosity improver to control the viscosity of the oil at different temperatures. A research scientist will experiment with different oils by running engines containing the oils and analyzing the effect of the engines on the oils. One of the important analysis and design tools we have is the break-down of complex objects into their constituent parts. It provides a meaningful and sensible decomposition, and it provides scope for re-use of components. Further, the constituent components are often easier to design than large, complex components - this is the thesis on which the early ideas of structured programming were based.

Delegation

The behavior of an object which is made up of other objects is usually defined by the behavior of the smaller objects. For example:

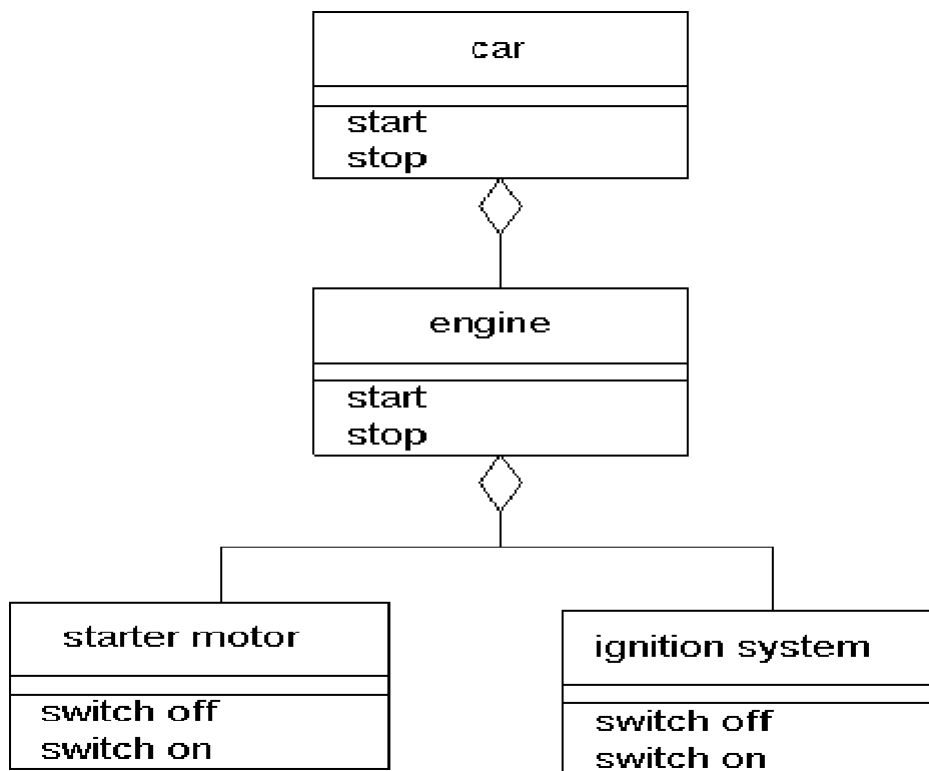


Figure 3.6: Delegation Example

To start a car, you start the engine. Thus the start operation on the car involves calling the start operation on the engine. This is known as delegation. The engine then will switch on the ignition system, switch on the starter motor then switch off the starter motor. This is further delegation. To stop the car, there will be a call to stop the engine, which in turn will make a further call to switch off the ignition. You may read elsewhere about the benefits of multiple inheritance. Most of the features of multiple inheritance can be simulated using delegation, with safer consequences. However, the arguments for and against multiple inheritance (inheriting from more than one parent) are lengthy and can be side-stepped for now.

Relationships

Objects can be related in other ways than by inheritance and aggregation. Any relationship between real world objects can be modeled: cats eat canaries, dogs bite postmen, the woodcutter murders the wolf, cars run over little old ladies, employees work for organizations, patients visit hospitals, patients stay in hospitals.

One to one relationships

In a one-to-one relationship, one object is associated with exactly one of its related objects. This is modeled by a straight line drawn between the objects. If the relationship is one-way, then an arrow is used to indicate the direction. The line can be labeled.

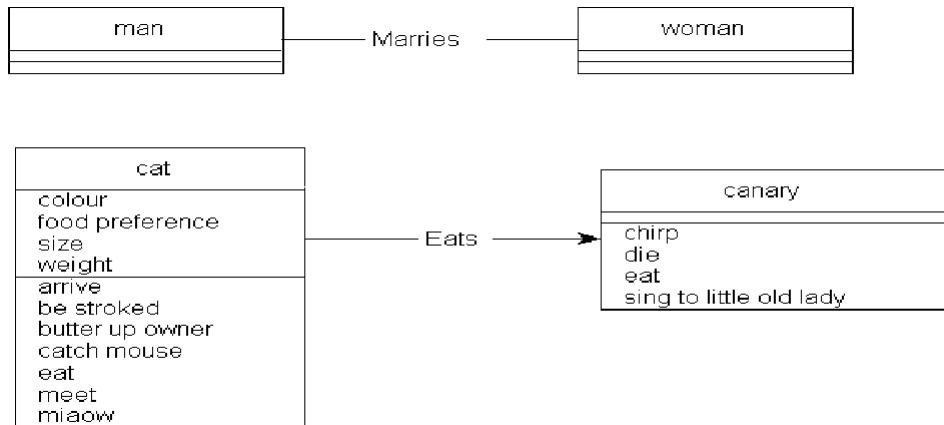


Figure 3.7: One to One Relationship

Thus a man marries one woman (at a time) and a woman marries one man (at a time). A cat eats one canary (before being battered to death by the little old lady who owned the canary). Canaries do not (in general) eat cats, so the eats relationship is one way.

One to Many Relationships

Sometimes one object can be related to many objects. This is indicated by different marks at the end of the line.

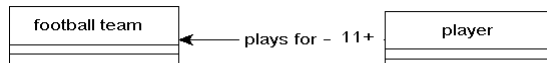


Figure 3.8: One to Many Relationship

A player plays for one football team. There are at least 11 players for a given football team. Football teams do not play for players.



Figure 3.9: One to Many Relationship

Zero or more suitors court the Princesses. The black dot at the end of the line indicates zero or more.



Figure 3.10: One to Many Relationship

Before an adequate suitor comes along, as is well known, a princess will kiss at least one frog, and possibly many more if she gets a taste for them. Frogs, being well mannered creatures and not wishing to appear in the gossip columns, never let more than one Princess kiss them.

Many to Many Relationships

Sometimes objects at either end of a relationship may be related to many objects at the other end.

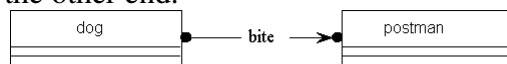


Figure 3.11: Many to Many Relationship

A dog may bite zero or more postmen. A postman may be bitten by zero or more dogs.



Figure 3.12: Many to Many Relationship

A lubricant is recommended for at least one engine. An engine has at least one lubricant recommended for it.

3.4. Object Models

We now have the notation to describe quite complicated systems. The process of object-oriented analysis and design is one of elaborating an object model, increasing its detail and scope until enough is known to construct a computer system (if indeed that is what is wanted).

An example is one to describe patient referrals by GP's to specialists.

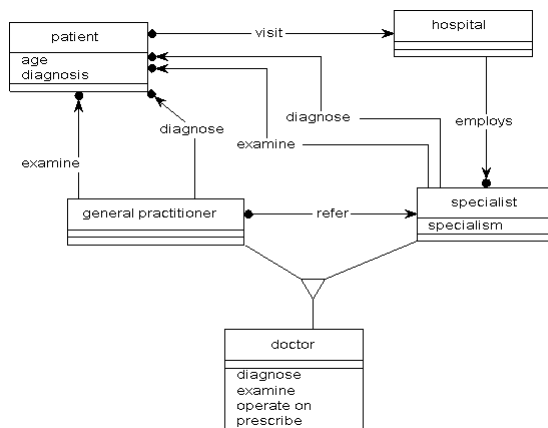


Figure 3.13: Object Model

Another example is for a lift system:

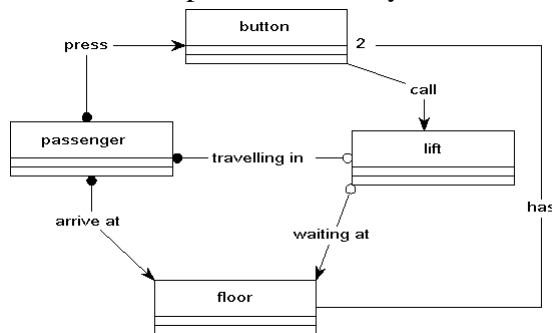


Figure 3.14: Object Model

The object model is the principal output of an analysis and design process. The distinction between analysis and design is much greyer in object-oriented development. Essentially it is one of detail. Analysis usually omits concerns about how a system is to be developed, and some of the objects may not be fully decomposed. The object model is the central pillar of a analysis or design. It defines the computable elements.

Learning Activities

Construct an object model for one of the following: the football league, star wars, a personnel system, Cinderella, an automatic washing machine, a video hire shop, a kitchen.

Revision Questions

EXERCISE 5.  Briefly explain why it is important to categorise objects into classes

Example  . Differentiate between include and extend relationship for use cases

Solution: for revision



EXERCISE 6.  Briefly explain the concept of an object and hence describe three characteristics of an object.

References and Additional Reading Materials

1. TCAD Journal: The Journal of Technology, Computer Aided Design.
 - <http://www.ieee.org/journal/tcad/>
2. CAD/CAM Publishing: books, magazines, industry links <http://www.cadcamnet.com/>

LESSON 4

Requirements Capture

Learning outcomes

Upon completing this topic, you should be able to:

1. Explain the importance of requirements capture.
2. Outline challenges faced in requirements capture.
3. Outline the sources of requirements.
4. Describe and carry out the elicitation process.

4.1. Requirements Capture

Part of the job of the system analysts is to find out from the users what they require in an information system. Indeed, identifying what a new system should be able to do is one of the first steps in development. The aim of developing a new information system must be to produce something that meets the needs of the people who will be using it.

Goguen defines requirements as properties that a system should have in order to succeed in the environment in which it will be used.

Sommerville and Sawyer define requirements as descriptions of how the system should behave, or of a system's property or attribute. Requirements are also the knowledge of the application domain and constraints on the operation of the system. Software projects are all too often undertaken without having a good understanding of the needs or wants of the client. Often projects fail because users' requirements are not accurately defined.

Boehm and Davis state that, the requirements phase of a large system development project is the most error prone, and these errors are the most expensive to correct. Consequently, this phase has the greatest economic leverage. Further, if the requirements are wrong, Sommerville says that the system may be delivered late and the cost more than originally expected. He adds that if the system continues its use, the cost of maintaining and evolving the system are very high. A basic question in Requirements Engineering is to find out what users really need.

As discussed in the previous chapter, many large projects fail because of inadequate requirements gathering. Requirements capture involves determining all of the sources of requirements and then using these to understand the problem to be solved and the characteristics of the solution. Correct requirements build quality into the product from the start. Attempting to test quality into code at the end of a project is expensive and ineffective. The probability that, defects originating from defective requirements will be completely redefined, decreases as they are allowed to propagate into the design code.

Good requirements promote agreement among developers and users, on the job to be done and the acceptance criteria for the delivered system. Developing requirements also provides a sound basis for resource estimation (cost, personnel quantity and skills, equipment, time). Improving one's organization competence in requirements capture will apply limited resources at the point of leverage to increase

customer satisfaction and development team productivity. Brooks states that; ‘The hardest single part of building a software system is deciding precisely what to build. No other part of the conceptual work is as difficult as establishing the detailed technical requirements, including all the interfaces to people, to machines, and to other software systems. No other part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify late’

Collecting requirements is not an easy task because of several reasons:

1. A large number of requirements must be taken into account. For scheduling problems that have many constraints and may also have several stakeholders, several requirements normally arise.
2. Because of different stakeholders and different needs, several requirements are bound to arise which contradict each other, and need to be prioritized.
3. Requirements are inherently imprecise and often incomplete.
4. Implications of requirements and feasibility are hard to predict.
5. Different stakeholders with different backgrounds, interests and views are involved in the requirements development process.
6. Requirements change constantly.

In as much as it is a challenging task developing requirements for a system, this does not compare with the cost of not developing requirements. For example, lack of proper requirements development was not the only cause, but it was a substantial contributor to the failure of the London ambulance service computer aided dispatch system.

4.2. Sources of Requirements

1. Stakeholders

A system stakeholder is anyone who benefits in a direct or indirect way from the system that is being developed (Sommerville and Sawyer, 2004). That is, stakeholders are those people or groups that will in some way, be affected by the system.

1. Application Domain

The application domain places domain constraints, which generate system requirements and may place limitations on other requirements. The timetabling problem is one that has been researched upon numerously and some general constraints are expected of class timetabling systems.

1. The existing system

The need for an automated system could be born from the fact that currently a manual system is being used and a computer is used only for typing and printing. Also, there could be a need to replace an old computer system with latest technology. The existing system has to be understood so as to facilitate developing requirements for a new computerized system. It also aids in understanding why the manual system is being used and why there is need for a computerized system. The existing system also provides documents that are examined and are used as a source of requirements.

4.2.1. Stakeholder Identification

As part of the requirements capture process, all potential stakeholders should be explicitly defined and consulted to discover if they have specific requirements or if they impose specific constraints on the system.

The reasons for careful consultation of stakeholders include:

1. If everyone who is likely to be affected by the system is not considered, important requirements are likely to be missed and the system may have to be reworked to include these at a later stage or, in the worst case, may not be used.
2. Identification of stakeholders can help in discovery of viewpoints for structuring the system requirements. This is especially important if a viewpoint oriented approach to structure the requirements elicitation process is used. Viewpoints are a way of structuring the requirements to represent the perspective of different stakeholders.
3. Identifying stakeholders and discussing the system with them, makes them feel that they are part of the development process. They are more likely to be sympathetic to the introduction of the system and to volunteer information about their requirements.

Sommerville and Sawyer (2004), suggested that the following checklist be used in identifying stakeholders:

1. By discovering the potential end-users of the system and the customers of the organization who will use the system. Determining the needs of users will involve understanding the activities certain users are performing in order to accomplish their work (Jirotko and Lincoln, 2000).
2. By initial discussion with organizational management and by considering descriptions of the business processes that the system is intended to support, and the people involved.
3. By considering engineers and maintenance staff responsible for developing and maintaining the system.

Example Stakeholders:

- Users: concerned with the features and functionality of the new system
- Designers: want to build a perfect system, or reuse existing code
- Systems analysts : want to “get the requirements right”
- Training and user support staff: want to make sure the new system is usable and manageable
- Business analysts: want to make sure “we are doing better than the competition”
- Technical authors: will prepare user manuals and other documentation for the new system
- The project manager: wants to complete the project on time, within budget, with all objectives met.
- “The customer”: Wants to get best value for money invested!

4.2.2. The Elicitation Process

The elicitation process focuses on understanding the users’ needs and how to communicate these effectively. There are several elicitation techniques available and the general consensus is that there is no ‘best’ one.

Introspection

Introspection involves imagining what kind of system a user would want if they were to use an automated or new system. Introspection is a useful method but since it's biased towards one's personal opinion, its results cannot be entirely relied upon and need to be used in conjunction with the results from the other techniques.

Interviews

Open-ended and/or unstructured interviews can be used; Open-ended interviews are more likely to obtain unrehearsed and spontaneous information, while unstructured interviews allow the interviewee to be more at ease and talk informally about the questions asked.

Questionnaires

Questionnaires can be used to elicit requirements in situations where the anonymity of the respondents is required or just to gain more information from the users. Participation/Observation This involves observing the way users interact with an existing system and also taking part in its use and/or creation.

Prototyping

A simple throwaway prototype can be used to elicit additional requirements. The prototype may or may not employ an algorithm to implement the system, but rather simply provides a possible graphical user interface of the proposed system.

4.2.3. Starting Points for Elicitation

1. Boundaries
 - Scoping the problem
2. Stakeholders
 - Identifying the problem owners
3. Goals
 - Identifying the success criteria

4. Scenarios

- Using concrete examples to understand the problem

4.2.4. Where do we Start?

1. Identify the problem

- what is the objective of the project?
- the “vision” of those who are pushing for it? e.g., “Scheduling meetings is too costly right now”

1. Scope the problem

- given the vision, how much do we tackle? e.g. “Build a system that schedules meetings”, ... or ... e.g. “Build a system that maintains people’s calendars” ... or ...

2. Identify solution scenarios

- given the problem, what is the appropriate business process for solving it? e.g. “Anyone who wants to schedule a meeting goes to the secretary, gives details and the secretary handles the rest”, ... or ...

3. Scope the solution

- Given a business process, what parts should be automated, and how? e.g. “Computer takes in scheduling request details, outputs a solution” ... or ... e.g. “Solution arrived at interactively by secretary and computer” ... or ...

4.2.5. Starting Point

1. Some notion that there is a “problem” that needs solving

- e.g. dissatisfaction with the current state of affairs
- e.g. a new business opportunity
- e.g. a potential saving of cost, time, resource usage, etc.

2. Collect enough information to:

- identify the “problem”/”opportunity”
 - Which problem needs to be solved? (identify problem Boundaries)
 - Where is the problem? (understand the Context/Problem Domain)
 - Whose problem is it? (identify Stakeholders)
 - Why does it need solving? (identify the stakeholders’ Goals)
 - How does the problem manifest itself? (collect some Scenarios)
 - When does it need solving? (identify Development Constraints)
 - What might prevent us solving it? (identify Feasibility and Risk)
 - Become an expert in the problem domain
 - Learn how to find your way round a new problem area quickly
 - Use your (initial) ignorance as an excuse to ask (dumb?) questions
 - Recognize the domain expertise of the people you talk to

3. Vague problem stated by the customer:

- E.g. university textbook store:
 - Manager wants to computerize the book order forms filled out by instructors;
- E.g. A large insurance company:
 - Claims manager wants to cut down the average time it takes to process an insurance claim from 2 months to 2 weeks
- E.g. A telecommunications company:
 - CIO wants to integrate the billing system with customer record systems of several affiliates, so there is only one billing system...
- E.g. Large Government Aerospace Agency:
 - The president wants to send a manned mission to Mars by the the year 2020

1. Often you only see symptoms rather than causes:
 - E.g. “JKUAT patients needing X-ray scans have to wait for months”
 - The long wait is the symptom, not the problem. The problem may be:
 - * Shortage of X-ray machines;
 - * Shortage of trained staff;
 - * Shortage of doctors to process the data
 - * Inefficient scheduling procedures

4.3. Problems Faced While Eliciting Requirements.

A good requirements elicitation process supports the development of a specification that is unambiguous, complete, verifiable, consistent, modifiable, traceable, and usable, to name but a few. However, problems with requirements elicitation inhibit the definition of such requirements.

1. Stakeholders often do not really know what they want from the computer system except in the most general terms. Even when they have a clear idea of what they would like the system to do, they often find this difficult to articulate. They may make unrealistic demands because they are unaware of the costs of their requests. This was dealt with by eliciting information about the tasks users currently perform and those that they may want to perform. And also recognizing the users as domain experts and employing different elicitation techniques.
2. The ‘yes, but’ syndrome. It is human nature to either like something very much or to like something but to find faults in it or to suggest ‘what could be better’. This problem was dealt with by employing techniques that bring out the ‘yes, but’ early enough and also plan for time to get feedback from users.
3. The requirements engineer may not know when to stop eliciting for requirements. This was dealt with by scoping the requirements effort and defining the problems that are to be solved by the system.
4. The stakeholders may not understand terms that the requirements engineer is using and may give answers not required. For example, from the interview

with the timetabling master, when asked about constraints, the timetabling master spoke of the problems they faced.

5. The large amount of information gathered during elicitation necessitates that it be structured in some way. The understanding of this structure is dependent on the characteristics of the stakeholder communities.
6. The language used to express the requirements back to the stakeholder communities may be too formal or too informal to meet the needs of each of the groups.
7. Requirements often change. During the time it takes to develop a system the users' needs may mature because of increased knowledge brought on by the development activities, or they may shift to a new set of needs because of unforeseen organizational or environmental pressure. Requirements may emerge from new stakeholders who were not originally consulted. Additionally, the economic and business environment in which the analysis takes place is dynamic. It inevitably changes during the elicitation process. Hence the importance of particular requirements may change.
8. Some users did not respond to the questionnaires.

A major goal in Requirements Engineering is to minimize such problems. This has involved putting significant effort into requirements elicitation. This process is not a one-off procedure but an iterative process that goes on throughout the requirements gathering process.

Learning Activities

Carry out requirements capture in the organization that you are working in. This assignment has ten steps as outlined below:

1. Identify a problem within the organization you are working with that is amenable to requirements engineering techniques.
2. Scope the problem, i.e., choose how small or large a problem you will tackle during your feasibility study.
3. Interview key people involved in the problem. These may include your contacts, plus others with responsibilities related to the feasibility study.
4. Study relevant documents. This may involve reading policy documents, memos, documentation on current systems and new objectives/needs.
5. Define alternatives for conducting business and for automation. Define the criteria you will use to evaluate and choose among these alternatives. Your criteria should include cost/benefit, where applicable.
6. Create models of the key aspects of the problem and the alternative solutions. These should include:
 - (a) Goal models to represent the relationships between goals of the key stakeholders;
 - (b) Activity diagrams and/or business process models to represent important business processes related to the problem and the alternative solutions.
7. Assess unusual circumstances or special attention items. This may involve special requirements for particular employees or customers of the organization.
8. Evaluate the alternatives on technical, economic, schedule and operational feasibility, and choose the one that looks most promising.
9. Write a report that describes the objectives of your study, the problem you identified the alternatives you explored, as well as your recommendations.

Revision Questions

Example . Briefly describe the phases of the UML process

Solution: for revision



EXERCISE 7.  State the normal procedure for identifying the following, from a textual description. (a) Objects (b) Operations

EXERCISE 8.  Define the term UML, explaining its difference from other types of Object Modeling Techniques (OMTs)

References and Additional Reading Materials

1. J W. Satzingr, Robert B.Jackson, Stephen D. Burd (2004). Object oriented analysis and Design with Unified Process, ISBN-13: 978-0619216436
2. Edward Coad (1990). Object Oriented Analysis, ISBN-13: 978-0136299813

LESSON 5

UML Diagrams

Learning outcomes

Upon completing this topic, you should be able to:

1. Describe the systems behavior from the user's point of view.
2. Explain and use the concept of Use case diagrams.
3. Draw use case diagrams of the requirements captured in the previous week.

5.1. The Unified Modeling Language (UML)

UML is a language for specifying, constructing, visualizing, and documenting the artifacts of a software-intensive system.

First and foremost, the Unified Modeling Language fuses the concepts of Booch, OMT, and OOSE. The result is a single, common, and widely usable modeling language for users of these and other methods.

Second, the Unified Modeling Language pushes the envelope of what can be done with existing methods. As an example, the UML authors targeted the modeling of concurrent, distributed systems to assure the UML adequately addresses these domains.

Third, the Unified Modeling Language focuses on a standard modeling language, not a standard process. Although the UML must be applied in the context of a process, it is our experience that different organizations and problem domains require different processes. (For example, the development process for shrink-wrapped software is an interesting one, but building shrink-wrapped software is vastly different from building.

UML is a Language for Specifying, Visualizing, Constructing, and Documenting Software Artifacts. A Modeling Language Provides the following:

1. Model Elements: Concepts and Semantics
2. Notation: Visual Rendering of Model Elements
3. Guidelines: Hints and Suggestions for Using Elements in Notation

The primary design goals of the UML are as follows:

1. Provide users with a ready-to-use, expressive visual modeling language to develop and exchange meaningful models.
2. Furnish extensibility and specialization mechanisms to extend the core concepts.
3. Support specifications that are independent of particular programming languages and development processes.
4. Provide a formal basis for understanding the modeling language.

5. Encourage the growth of the object tools market.
6. Support higher-level development concepts such as components, collaborations, frameworks and patterns.
7. Integrate best practices.

In terms of the views of a model, the UML defines the following graphical diagrams:

- use case diagram
- class diagram
- behavior diagrams:
 - statechart diagram
 - activity diagram
 - interaction diagrams:
 - * sequence diagram
 - * collaboration diagram
- implementation diagrams:
 - component diagram
 - deployment diagram

UML Diagrams and their Descriptions

The following table describes the UML diagrams in use. In this unit, we shall consider a few of them.

Diagram	Description
Activity Diagram	Depicts high-level business processes, including data flow, or to model the logic of complex logic within a system.
Class Diagram	Shows a collection of static model elements such as classes and types, their contents, and their relationships.
Communication Diagram	Shows instances of classes, their interrelationships, and the message flow between them. Communication diagrams typically focus on the structural organization of objects that send and receive messages. Formerly called a Collaboration Diagram.
Component Diagram	Depicts the components that compose an application, system, or enterprise. The components, their interrelationships, interactions, and their public interfaces are depicted.
Composite Structure Diagram	Depicts the internal structure of a classifier (such as a class, component, or use case), including the interaction points of the classifier to other parts of the system.
Deployment Diagram	Shows the execution architecture of systems. This includes nodes, either hardware or software execution environments, as well as the middleware connecting them
Interaction Overview Diagram	A variant of an activity diagram which overviews the control flow within a system or business process. Each node/activity within the diagram can represent another interaction diagram.
Object Diagram	Depicts objects and their relationships at a point in time, typically a special case of either a class diagram or a communication diagram.
Package Diagram	Shows how model elements are organized into packages as well as the dependencies between packages.
Sequence Diagram	Models the sequential logic, in effect the time ordering of messages between classifiers.
State Machine Diagram	Describes the states an object or interaction may be in, as well as the transitions between states. Formerly referred to as a state diagram, state chart diagram, or a state-transition diagram.
Timing Diagram	Depicts the change in state or condition of a classifier instance or role over time. Typically used to show the change in state of an object over time in response to external events.
Use Case Diagram	Shows use cases, actors, and their interrelationships.

Figure 5.1: UML Diagrams**5.2. System's Behavior from User's Point of View: Use Case Diagrams**

To model the system's behavior from the users' point of view, a use case model is used. A use case model is an abstraction of the problem domain and helps to facilitate well articulated and unambiguous understanding of the problem domain. A use case template is used to detail the use case model. Use case models involve actors and use cases. An actor is something that interacts with the system. A use case model captures the problem domain in terms of functionality provided (use cases), and the 'roles' (actors) for whom these functions are performed; the actors are represented by stick figures and the use cases by ovals. An actor makes use of a system in different ways. Each of these ways is known as a use-case. A use-case may involve a number of actors, just as an individual actor may make use of several use-cases.

In summary, a use case diagram shows the relationship among use cases within a system or other semantic entity and their actors.

Semantics

Use case diagrams show actors and use cases together with their relationships. The use cases represent functionality of a system or a classifier, like a subsystem or a class, as manifested to external interactors with the system or the classifier.

Notation

A use case diagram is a graph of actors, a set of use cases, possibly some interfaces, and the relationships between these elements. The relationships are associations between the actors and the use cases, generalizations between the actors, and generalizations, extends, and includes among the use cases. The use cases may optionally be enclosed by a rectangle that represents the boundary of the containing system or classifier.

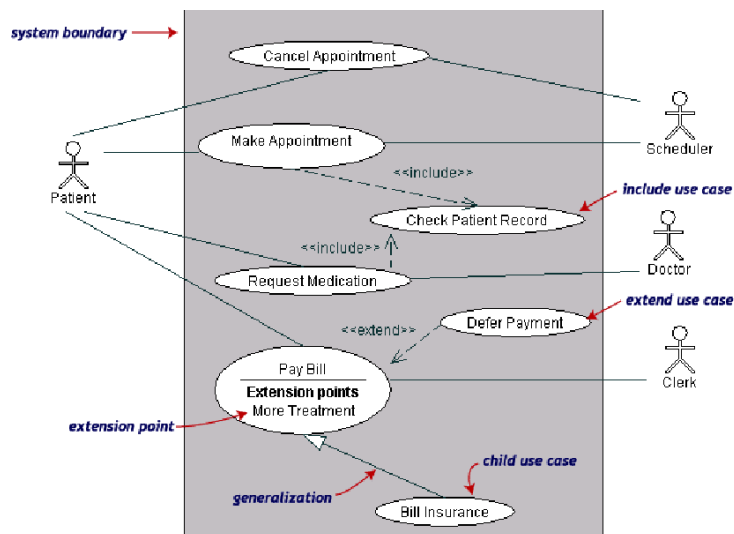


Figure 5.2: Use Case Diagram Indicating System Boundary

5.2.1. Use Cases for Requirements Capture

Use cases determine the system requirements by focusing on how the system is used. This is not really new – for a long time, people have used typical interactions with a system to help understand requirements. These have tended to be excessively informal and rarely documented.

Ivar Jacobson popularized this approach in the early 1990s in his Objectory method. Interactions with the system are captured by use cases (diagrammatically and textually). These are more structured/stylised than the previous ad hoc approaches. This is now incorporated in the Rational Unified Process (RUP).

5.2.2. Use Cases and Scenarios

A scenario is one possible interaction with a system, e.g. The customer browses the catalog and adds desired items to the shopping basket. When the customer wishes to pay, the customer describes the shipping and credit card information and confirms the sale. The system checks the authorization on the credit card and confirms the sale both immediately and with a follow-up email. (Fowler).

There could be many variations on the above:

- .e.g. the customer may change their mind about the purchases
- .e.g. the credit card authorization may fail

The set of related scenarios is called a **use case**, i.e. a use case is a pattern of interaction with a system

5.2.3. Specifying a Use Case

A use case is often specified as a sequence of numbered steps with alternatives as variations on that sequence, e.g.

Buy a Product

1. Customer browses through catalog and selects items to buy
2. Customer goes to checkout
3. Customer fills in shipping information
4. System presents full pricing information, including shipping
5. Customer fills in credit card information
6. System authorizes purchase
7. System confirms sale immediately
8. System sends confirming email to customer

Alternative: Authorization failure

- At step 6, system fails to authorize credit purchase
- Allow customer to re-enter credit card information and retry

Alternative: Regular customer

- 3a. System displays current shipping information, pricing information,
 - and last four digits of credit card information
- 3b. Customer may accept or override these defaults
 - Return to primary scenario at step 6

5.2.4. Use Case Diagrams

Use case diagrams identify the use cases and the entities outside the system that will interact with them.

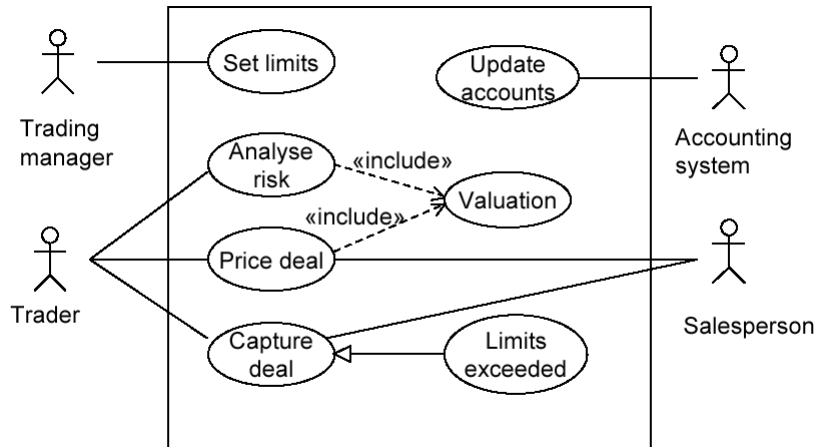


Figure 5.3: Use Case Diagram with Actors

The use case diagram has the following:

1. 4 actors (or external entities)
2. 7 use cases (or patterns of interaction)
3. 5 use cases directly accessed
4. 2 use cases indirectly accessed via relations with other use cases.

The diagram is a visual summary of a number of interactions. More detailed descriptions (as indicated before) are also required.

5.2.5. What is an Actor?

An actor is a role that some external entity might play on. We are not concerned with which particular entity is involved. We are not concerned with whether the one entity may play multiple roles, i.e. may interact with the system as different actors.

- An actor doesn't have to be a person even though a stick figure is used on. e.g. accounting system.
- An actor should be a beneficiary of the system.
 - The functionality provides some benefit to the actors

- This will give a natural association which will be retained (possibly in modified form) during the development of the system
- Cockburn refers to the primary actor, the one initiating the interaction
- There are three basic approaches:
 - always show interaction with external systems
 - show interaction with external systems when they initiate contact
 - show interaction with external systems when they derive benefit

Examples

In a banking system where a customer can withdraw money, we would draw:

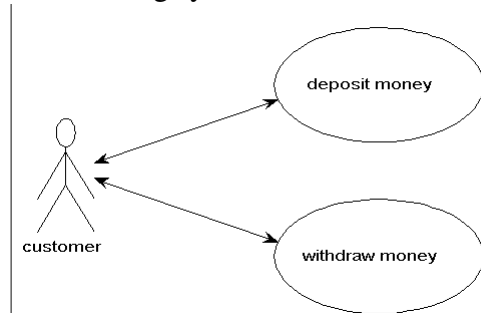


Figure 5.4: An Actor in a Banking System

Now it might be that the system which is being implemented in the bank needs to involve a cashier for depositing, but that to withdraw money the customer has to use the cash machine. The cashier is then an actor.

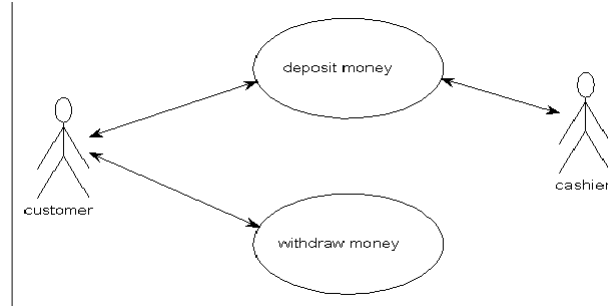


Figure 5.5: Two Actors in a Banking System

And so we build our model of the ways the system is used. We might add two more use-cases of the bank system.

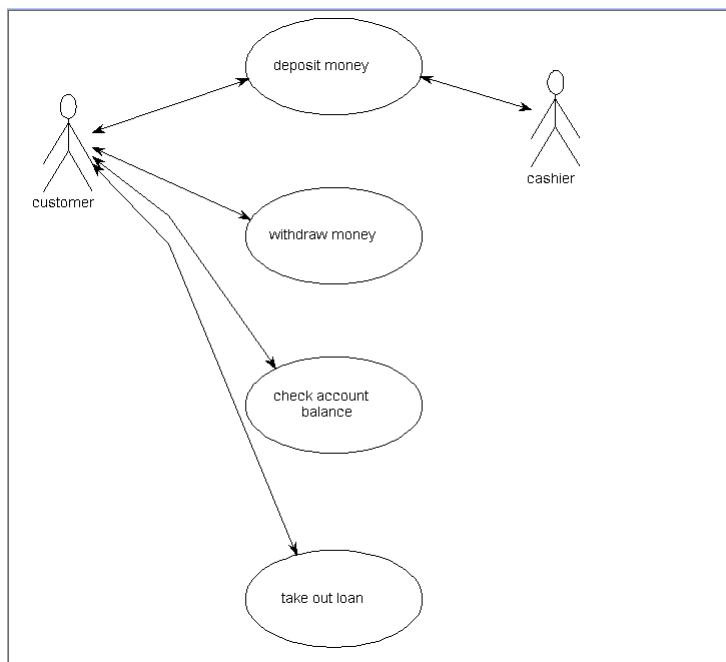


Figure 5.6: Banking System Use Cases

Examining this, we see that there is probably a need for another actor. (I never borrowed money without having to grovel to the bank manager, and the bank manager always looked carefully at my pitiful balance before grudgingly saying yes).

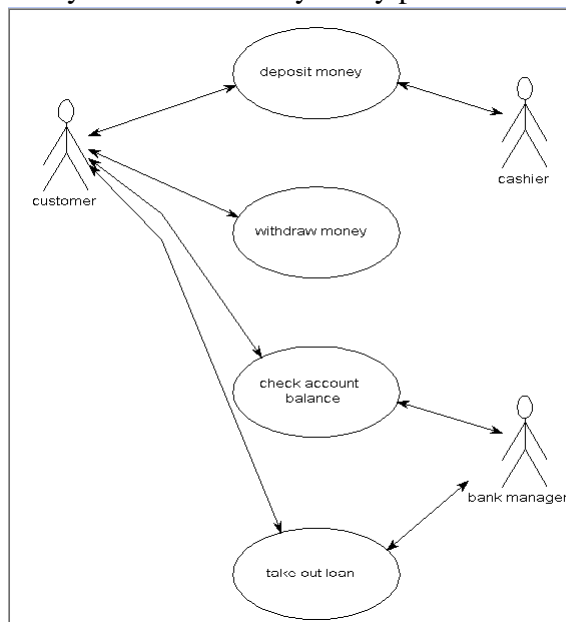


Figure 5.7: Banking System Use Cases with More Actors

Use cases may in fact use other use-cases. The withdraw cash use-case would make use of the account balance use-case before issuing the money.

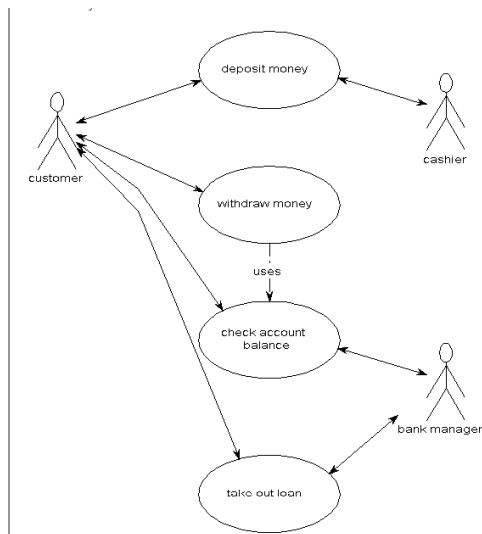


Figure 5.8: Use Cases using other use cases

A use-case is a very abstract view of what the system provides to the various actors who use it. It is not intended to give a detailed, blow-by-blow account of how the services are provided. Therein lies the beauty of the approach. By keeping out detail, it is possible to reason at a higher level. Later the use-cases can be opened out using object-interaction diagrams to begin to define their precise operation.

5.3. Associations Between Use Cases

Similarly, there can be associations between use cases. One use case may be part of another. Shown with the «includes» stereotype and a dashed arrow from the base use case to the one which is included.

Use Case Extensions

- Use cases can be extended by other use cases
- Usually, conditions exist somewhere in the use case's activity that would cause the extended use case to occur
 - This 'somewhere' is called an extension point in UML
- Use cases can be extended by other use cases
 - Usually, conditions exist somewhere in the use case's activity that would cause the extended use case to occur

- This 'somewhere' is called an extension point in UML
- The original use case is called the base use case o The new use case is called the extended use case
 - The extended use case usually augments the base use case with additional behaviour.
- For example, a use case for a hotel might be 'reserve hotel room'
 - An extension to this use case might be 'request coffee machine' to request that a coffee maker be in the room
 - The customer still wants to reserve a hotel room, but they also want a coffee maker in their room when they arrive
- Use cases extensions are specified using a dashed arrow
 - The arrow points from the extended use case to the base use case
 - The arrows are labelled with «extend» plus a description of the conditions which cause the extension to occur
 - The base use case usually contains a description of the extension points where the use case may be extended

Use Case Extension Example

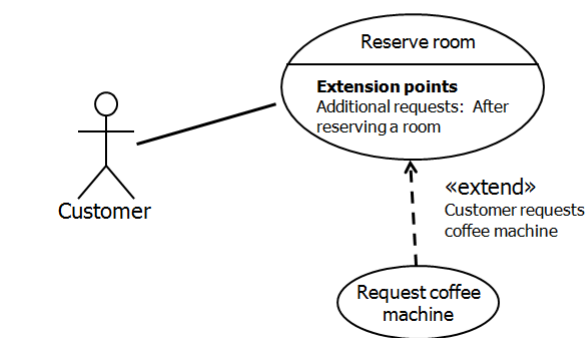


Figure 5.9: Use Cases Extension

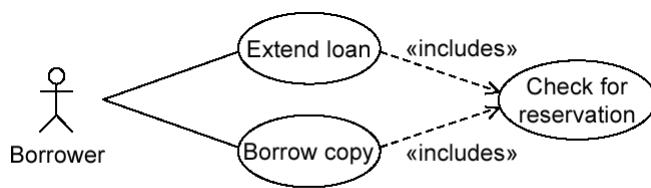


Figure 5.10: Use Cases Associations

- e.g. both analyse risk and price deal include valuation
- e.g. for a library system, both extend loan and borrow copy include check for reservation

One use case may be more general/specific than another. use the inheritance relation.

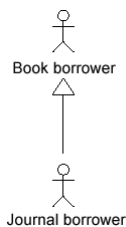


Figure 5.11: Use Cases Inheritance

- e.g. limit exceeded is a more specific use case than capture deal
- e.g. journal borrower is a more specific actor than book borrower Book borrower

Use Case Inclusions

Use case inclusion is when one use case is used within another use case

For example, the use cases 'withdraw' and 'deposit' in a bank, might be used within the use case 'transfer funds'.

Use case inclusion is specified using a dashed arrow. The arrow points from the container use case to the included use case. The arrow is labelled with «include».

Use Case Inclusion Example

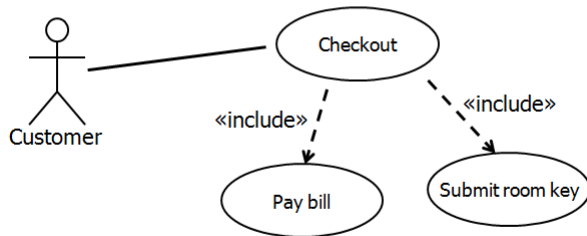


Figure 5.12: Use Cases Include

When there are significant variations of a use case

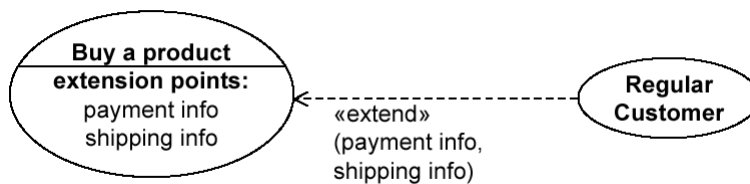


Figure 5.12: Use Cases Extends

Actor Inheritance

It may also make sense to have actor inheritance in a system, for example, an employee might have a specialization called manager. A manager might handle (in addition to normal employee duties) customer complaints, refunds, scheduling, etc.

Actor Inheritance

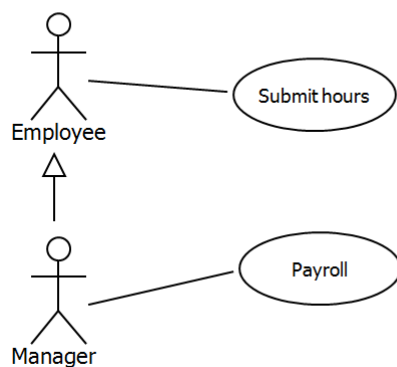


Figure 5.13: Actor Inheritance

5.4. Summary

1. Use case diagrams consist of at least two components:
 - (a) An actor: represented as stick people, with a label below, naming the actor's role. There may be multiple actors in a diagram
 - (b) A use case: Use cases are represented as ellipses, with a label inside, naming the use case. There may be multiple use cases in a diagram.
2. For an example, consider an auction website (e.g. eBay)
 - (a) The actor customer might:
 - i. Register for an account
 - ii. Search for items by keyword
 - iii. Place a bid on an item
 - iv. Put an item up for auction
 - v. Check the status of this itemAn actor represents a role that a user might play.
3. Use cases are not a good tool for finding classes because:
 - (a) Use cases emphasize ordering ("When a customer places an order over the phone, his credit card number is validated. Then the database is updated and a confirmation number is issued", etc.). This is incompatible with object technology: the method shuns early reliance on sequentiality properties, because they are so fragile and subject to change.
 - (b) Relying on a scenario means that you focus on how users see the system's operation. But the system does not exist yet. (A previous system might exist, but if it were fully satisfactory you would not be asked to change or rewrite it.)
 - (c) Use cases favour a functional approach, based on processes (actions). This approach is the reverse of O-O decomposition, which focuses on data abstractions; it carries a serious risk of reverting, under the heading of object-oriented development, to the most traditional forms of functional design.

Learning Activities:

1. Travel Agent Application

Initial requirements description:

1. The desired system is an application which allows travel agent to customize trips for customers
2. Enter the requirements analysis phase
 - (a) We should add anything missing from the requirements capture phase
 - (b) We should clear up any ambiguities or poorly worded phrases
3. Requirements Analysis:
 - (a) What features do we need?
 - i. We need the ability to create a customized trip for the customers
What does this involve?
 - ii. To answer this question, we need to converse with the customers and travel agents. We can't actually do that here, so let's just make some decisions based on what we think would be appropriate:
 - iii. Obviously, in the end, designers and developers would never be the people to make these decisions
 - iv. Assume that the following decisions were achieved through an interaction between customers, travel agents, and architects Customizing a Trip
 - A. Customers will want to o Select the flights that suit them
 - B. Based on price, class, times, etc.
 - C. Choose how many days to stay
 - D. Choose their hotel & room type
 - E. Based on price, class, amenities, etc.
 - F. Add any extras to the trip
 - G. Excursions to an attraction, rental of a car, a city tour, etc.
 - (b) Use Cases

- i. So what are our use cases?
 - A. Customers booking a trip is one
 - B. Cancelling trips might be another
 - C. Let's keep it simple, and stick to these 2 for now
- ii. So who are our actors?
 - A. Customers
 - B. Travel Agents
- iii. Which actors participate in each use case?
 - A. Customers and Agents both participate in both use cases

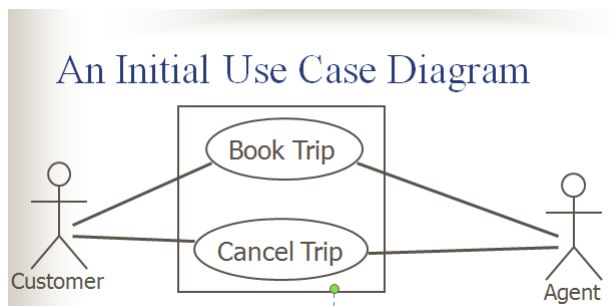


Figure 5.14: Initial Use Case Diagram

Requirements analysis occurs in a few stages:

- Identify and clarify use cases
 - Create an initial use case diagram
- Identify any use case special cases
 - Update the use case diagram to include use case extensions to accommodate these special cases
- Elaborate on use cases
 - Update the use case diagram to include any behavioural inclusion (i.e. behaviour components).
 - This typically involves the use of use case inclusion.

Additional Information

- Booking a trip should be the same for all customers, with one exception:
 - If the customer wants to book some trip extras, there will be some extra work to be done by our program
 - This could include making car rental reservations, or ordering tickets to excursions, attractions, or tours
- What UML feature should be use to represent this optional added behaviour?

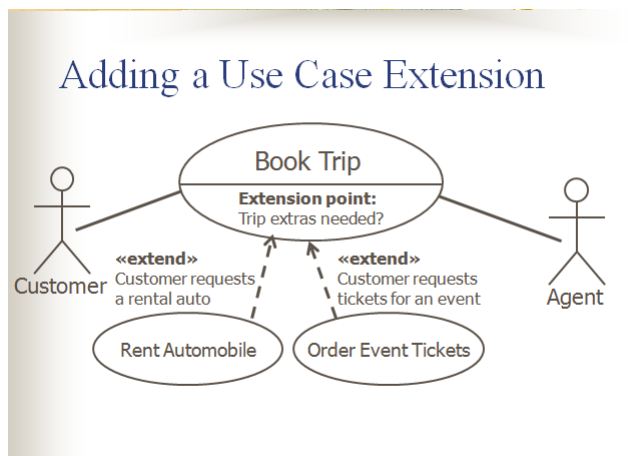


Figure 5.15: Initial Use Case Diagram with Extensions

Use Case Elaboration

- Book Trip
 - What is involved with this use case?
- Most trips include a flight and a hotel booking
 - So let's say that the Book Trip use case uses the following use cases:
 - * Book Flight.
 - * Book Hotel
- Cancel Trip
 - What is involved with this use case?

- We would need to cancel the flight, the hotel reservation, and the trip extras (if any)
 - Trip extras should be considered an extension, since it only occurs in some trips
 - Let's say the 'Cancel Trip' use case includes the following use cases:
 - * Cancel Flight
 - * Cancel Hotel Reservation

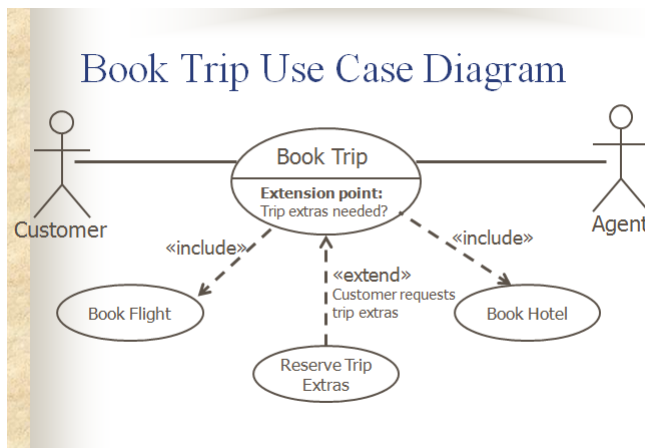


Figure 5.15: Book Trip Use Case

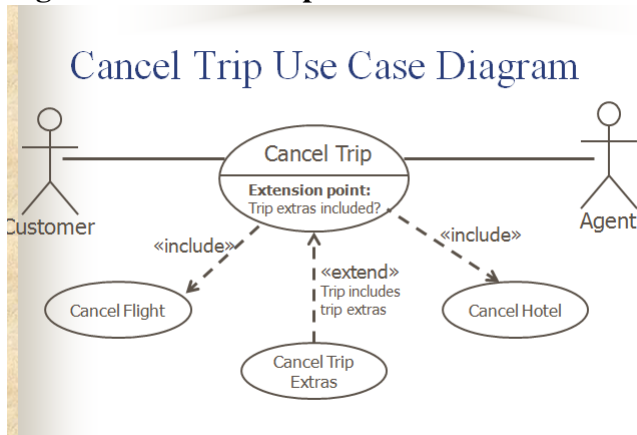


Figure 5.16: Cancel Trip Use Case

1. Draw a Use Case Diagram for the Class TimeTable System.

Revision Questions

Example . Explain the purpose of use case diagrams in object-oriented software development

Solution: for revision



EXERCISE 9.  In what ways might you expect end-users, clients and developers to disagree about problems in Information Systems Development?

EXERCISE 10.  Briefly explain FIVE steps involved in moving from the model you have developed to a detailed design of a software system supporting the Use Case

References and Additional Reading Materials

1. John. W Satzinger (2004). Object Oriented Analysis and Design with the Unified Process. Cengage Learning, ISBN-13: 978-0619216436
2. Acta Informatica ISSN 0001-5903
3. Advances in Computational Mathematics ISSN 1019-7168

LESSON 6

Requirements Analysis

Learning outcomes

Upon completing this topic, you should be able to:

1. Explain the different ways of classifying requirements.
2. Describe functional and non functional requirements.
3. Identify hard and soft constraints.

6.1. Requirements Analysis

Requirements analysis is the process of understanding the problem and the workable solution. It represents what is to be built and not how it is to be built and analyses users' wants and needs and communicates this in a language understood by the customer.

The unstructured collection of requirements are analysed and organized into coherent clusters such as functional and non-functional. Constraints and assumptions that would be considered in building the system are also considered. To understand the system context more, the domain is modelled at different levels of abstraction. Use case models have been used in the previous chapter to model the system's behaviour from the user's point of view and to define the system at an abstract level.

Class diagrams can be used to capture the structure of the domain and sequence diagrams can be used to capture the dynamic behaviour of the domain.

Use-cases allow us to describe at a high level the basic functionality of a system in terms of the various actors who need to interact with the system. We now consider how to flesh out those use-cases, and to link them into the object modelling method we considered earlier.

From the use-cases, which describe broad functionality, we begin to analyse the details of the functionality of the systems we want to construct. Before doing the domain modelling, we have to classify our requirements that have been captured.

6.1.1. Requirements Classification

The requirements discovered from the previous sections are now classified into functional, non-functional and constraints. Assumptions which are made are also discussed herein.

6.1.2. Functional Requirements

Functional requirements describe the intended behavior of the system and specify tasks the system is to achieve.

Sommerville (2005) state that functional requirements describe the functionality or behavior of the system: services, reactions to input, exceptions and modes of operation. These functional requirements have been captured in the use case model.

1. A user shall log into the system using a unique ID and password.
2. The system shall display an error alert if the log-in details are incorrect.
3. The user shall be able to log out.
4. The system shall be able to display the timetable for the whole week or for days or times specified by the user.
5. The system shall display the following information on the timetable.
 - (a) Time information.
 - (b) Days of the week.
 - (c) Course code.
 - (d) Classroom code.
6. The system shall be able to display the times for a course, for days or times specified by the user.
7. The system shall be able to display the room assigned to a course, for days or times specified by the user.
8. The system shall be able to display the title and surname of instructors assigned to a course.
9. The system shall be able to display free rooms, for days or times specified by the user.
10. The system shall display a message if there are no free rooms for the days or times specified by the user.
11. The system shall be able to display free times, for days or periods specified by the user.
12. The system shall display a message if there are no free times for the days or period specified by the user.
13. The system shall be able to display courses offered by respective departments.
14. The system shall be able to display courses respective student years.

15. The system shall be able to display the number of classes taking place at a time or day specified by the user.
16. The system shall provide the option for printing the queried timetable.
17. If the user opts to print the queried timetable, the system shall display an alert if no printer is connected to the computer.
18. The system shall have the option for saving to a disk, the queried timetable.
19. If the user opts to save the queried timetable to a disk, the system shall display an alert if the specified disk is not connected to the computer.
20. The system shall accept input data that is required for scheduling the timetable.
21. The system shall allow the user to update the new or changed data into the timetable.
22. The system shall display an alert if there is new data that has been input and no update has been made.
23. The system shall alert the user if there is data that has been changed and no update has been made.
24. The system shall display an alert for any data that is missing or conflicting, and is required to schedule the timetable.

6.1.3. Non- Functional Requirements.

Non- functional requirements are attributes that the system should have and constraints that are placed on the system.

The requirements are grouped into Quality requirements and Constraints.

Quality Requirements

The quality requirements used here are defined by ISO 9126. This is an international standard for software quality.

1. Functionality

Functionality is the essential purpose of a software system. The functionality of the class timetabling system is further defined under the following classes.

a) Suitability

- The timetabling system shall be used to schedule the class timetabling system at JKUAT
- The timetable should be created before the beginning of the respective trimester.
- The timetabling system shall allow more than one instructor to be assigned to a course.

1. Accurateness

- The timetabling system shall schedule classes with no conflicts.

1. Interoperability

- The system should run on the JKUAT LAN where it will be accessed from any workstation and terminal.

1. Security

- The different user groups shall have different security levels.
- Only the IT department will issue log-in details and can Change a user's security level.
- The log in details for the timetabling system shall be different from the log in details to the LAN.

b) Reliability

Reliability defines the capability of the system to maintain its service provision under defined conditions for certain periods of time.

1. Fault tolerance

- If a user enters their log-in details incorrectly, the timetabling system shall allow the user to re-enter their details as many times until they get it right.

- The system shall automatically log out the user after 10 minutes of inactivity.
- The timetabling system shall check for typographical errors.

1. Recoverability

- If the LAN connection is lost, once regained, the timetabling system should be able to operate as normal.

1. Availability

- Unless the LAN is down, the system shall be online 24 hours a day, seven days a week.

c) Usability

1. Understandability

- The system shall contain a comprehensive user manual.
- The system shall display information in a simple form style.
- The system shall use the 12-hr clock system.

1. Learnability

- Users of all levels of expertise should not take more than a day to learn to use the system.

1. Operability

- The system shall have a well-defined and easy to use graphical user interface.
- The user shall interact with the system using a keyboard and a mouse.

d) Efficiency

- The timetabling system shall respond to any user query in less than 10 seconds.
- The timetabling system shall respond to user queries 99% of the time.

- The system shall store data in a database system on the central server.

e) Maintainability

- The system shall be modular; hence there shall be little effort in applying changes to the system.

f) Portability

- The system shall be able to be used on PC and Macintosh platforms.

g) Design

- The system shall be developed using one or more platform- independent modular programming languages.

Constraints

Some constraints cannot be violated and have to be implemented into the system (hard constraints), while some can be violated but this violation should be avoided as much as possible (soft constraints). Therefore the constraints on JKUAT's class timetabling problem are divided into hard and soft.

Hard Constraints

1. The courses to be scheduled are undergraduate courses only.
2. Classes are scheduled on Monday to Friday only.
3. Times for scheduling are from 8 am to 6 pm
4. No classes are to be scheduled during lunch hour between 1 pm and 2 pm.
5. The class meetings of two different courses with the same student enrolment cannot be assigned to the same time slot.
6. A lecturer cannot teach more than one class meeting at the same time slot.
7. The assigned number of class meetings that are scheduled for the same time slot cannot exceed the number of available classrooms.
8. All class meetings should be assigned to a time slot.

9. All class meeting should be assigned to a room.
10. A room can only be assigned at most one class meeting at a time.
11. Course should be assigned to rooms with the necessary facilities.
12. Departmental constraints shall be implemented, for example, times for compulsory departmental meetings.
13. Time slot used is 1-hour for 2 credit hour courses, and 2 hours for 4 credit hour courses.
14. Courses should be assigned appropriate size rooms.
15. The Computer Laboratory 6 shall only be assigned practical courses of the Information Technology department.

Soft Constraints

These soft constraints are numbered in order of preference from high priority to lowest priority. During implementation, the scheduling algorithm may require that these constraints are weighted to determine which ones to consider first.

1. Where the number of students exceeds the appropriate room, more hours are allocated to the course.
2. Minimize consecutive classes for similar student groups or for the same instructor.
3. Departments should submit their preferences and constraints electronically.
4. There shall be minimal classes on Fridays afternoon.
5. There shall be minimal classes on Fridays in the afternoon
6. A lunch hour to be scheduled.

Assumptions.

Assumptions are conditions that must be true for use cases to terminate successfully.

The assumptions made while developing the above requirements for JKUAT are:

1. The departments assign and register courses to students.
2. The departments identify who teaches the courses.
3. The courses for credit transfer students are scheduled at the same time as the courses for regular students. That is, while departments forward courses to be scheduled, the courses to be offered to credit transfer students should be included.
4. The maximum number of classes that a student can attend in a day shall be specified.
5. The maximum number of hours that an instructor can teach in a day shall be specified.
6. The number of extra hours to be assigned to courses whose numbers exceed the capacity of the appropriate rooms, shall be specified.
7. User training shall be provided.
8. Unlike in the current system where individual departments forward complete timetables to the timetabling master for compilation, we shall assume that the departments shall only forward the following information to the timetabling master:
 - (a) Courses to be scheduled.
 - (b) Students registered for the courses.
 - (c) Number of students in the courses.
 - (d) Instructors assigned to the courses.
 - (e) Special room requirements such as computer lab, science lab etc.
 - (f) Special times requirements such as availability of part time lecturers.
 - (g) Departmental hard constraints, For example, no classes to be scheduled on Tuesday 2.00 pm to 4.00 pm for the computer science department since they have departmental meetings at that time.

6.2. Summary

Classifying requirements into the above groups assists in extracting external qualities of the system and the properties that these qualities shall have. Constraints define what requirements the system must fulfil for hard constraints and which ones it can fulfill if possible, for soft constraints. Assumptions are made to accommodate information that is needed for the requirements to be fulfilled.

Learning Activities

Identify all functional, non-functional requirements for the Travel Agent System.

Revision Questions

Example . What general advantages are claimed for the use of a methodology in the development of information systems?

Solution: for revision



EXERCISE 11.  What does it mean, to say that most modern methodologies are agile?

EXERCISE 12.  How might the following contribute to reuse in systems development? a) Encapsulation. b) Generalisation. c) Composition. d) Patterns.

References and Additional Reading Materials

1. CADALYST: A popular magazine focusing on AutoCAD. <http://www.cadonline.com>
2. An electronic CADD magazine (e-zine) focusing on the CADD industry news.
You can subscribe this weekly e-zine free of charge. <http://www.upfrontezine.com/>

LESSON 7

Domain Modeling

Learning outcomes

Upon completing this topic, you should be able to:

1. Explain the importance of domain modeling.
2. Describe the different models used.
3. Model the requirements from the use case diagram to all of the models examined.

7.1. UML Domain Modeling

Domain modeling is used to model the use cases. It is an abstract representation of a specification design, aiming to express the essentials of a situation without giving unnecessary information. The UML language is designed to express the static as well as the dynamic aspects of a system. The static part is represented mainly by the class diagram and defined by the classes, attributes and the relationships among classes. Sequence and class diagrams are discussed below.

7.2. Class Diagram

Class diagrams allow us to express classes in UML, including attributes and methods. Class diagrams allow us to declare our classes (visually). It is a UML diagram that shows classes with their attributes and operations together with associations between classes.

Classes in UML

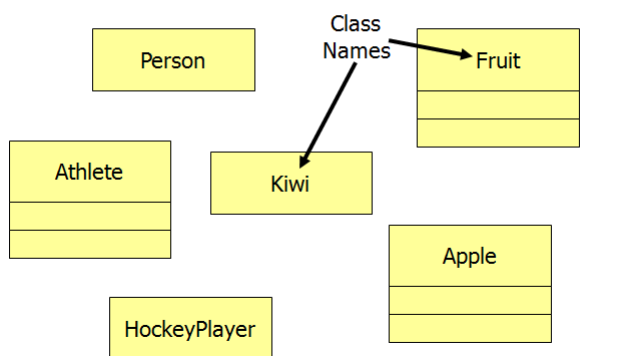


Figure 7.1: Classes in UML

As you can imagine, the class shapes shown here are a major part of class diagrams:

1. Both forms are acceptable for classes that have no attributes or methods.
2. The two empty blocks are used for holding declarations of attributes and methods.
3. The top block is used to hold the class name.

7.2.1. Adding Attributes

To find attributes, the real-world attributes of objects can be used as well as the attributes necessary to fulfill the requirements defined by the use case under consideration.

Attributes in Classes

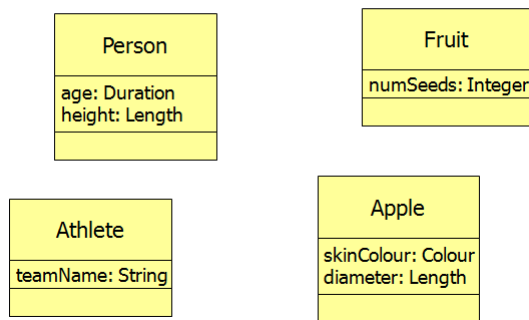


Figure 7.2: Attributes in Classes

Attributes are specified using the form:

- name: type.

This format is taken from Eiffel, which uses a Pascal-like syntax. Names are usually alphanumeric. Types are logical types from the domain. In this example, an apple's diameter is a quantity of type 'Length', which could be represented as a float or double in C++ or Java. Attributes can be read/write (default) or read only. Attributes that are read only are preceded with the modifier '/'. Attributes can be defined on the objects (default) or the class. Class attributes are preceded with the modifier '\$'. Attributes can have different visibilities: Public attributes, denoted with the modifier '+', can be accessed from outside the class. Private attributes, denoted with the modifier '-', can only be accessed from within the class. Protected attributes, denoted with the modifier '#', can be accessed from within the class, or any descendant (e.g. a subclass).

7.2.2. Operations

Operations can be defined, using the form:

- opName([in|out|inout] param1 : type1,...) : returnType

The modifiers in, out, and inout specify which direction(s) the parameters are to travel

- in: input to the operation
- out: output from the operation
- inout: input to the operation, possibly modified, then output from the operation

Often, if no modifier is specified, a parameter is assumed to be input only (in). Again, type1 and returnType are types from the application domain. Again, type1 and returnType are types from the application domain.

Abstract Classes and Methods

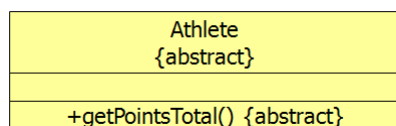


Figure 7.3: Abstract Class

To identify classes and create a class diagram:

1. list the candidate classes using the noun phrase identification related to the current requirements under consideration,
2. list the identified classes and draw them into the class diagram,
3. add the associations necessary to record relationships among classes,
4. add the attributes to fulfill the information requirements

7.3. Class Diagram Relationships

To satisfy the information requirement of the use cases under development, it is necessary to identify the associations between classes. A relationship is a semantic connection between objects of one or more classes. They are generally labeled with verb phrases. A relationship generally is described by its name, degree and multiplicity.

- Degree of a relationship
 - Number of classes that participate in that relationship

- The cardinality of object A is the number of objects of class A that can (or must) be associated with each object of class B
- Multiplicity in relationships (Multiplicity of object A)
 - The range of the number of objects of class A that can (or must) be associated with each object of class B
- There are three types of relationships:
 - A ‘is a type of’ B (inheritance)
 - A ‘is associated with’ B (association)
 - A ‘is a part of’ B (composition)

The technique of an association list is helpful for identifying relationships.

7.3.1. Class Diagram: Generalization

A superclass/subclass relationship based on inheritance. A general class (called the supertype) is modeled and then divided into several specialized classes (called subtypes). Inheritance is called “is-a” relationship.

Inheritance

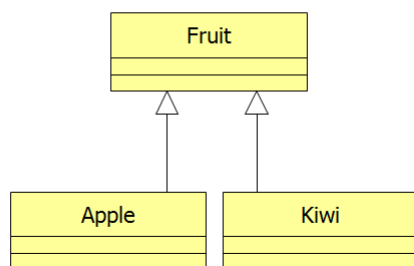


Figure 7.4: Inheritance Example

Inheritance

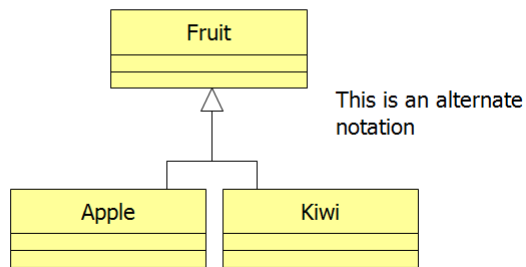


Figure 7.5: Another Inheritance Example
Multiple Inheritance

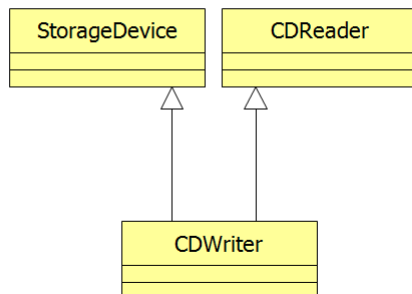


Figure 7.6: Multiple Inheritance

7.3.2. Groups

Whenever we create a subclass, we are defining a new group of objects that is a subset of the superclass' group. To be even more complete, when we define inheritance, we usually use keywords to give details about those groups; Overlapping/Disjoint

- Overlapping:
 - Groups are overlapping if there are objects that are members of both groups.
- Disjoint:
 - Groups are disjoint if there are no objects that are members of both groups.

- Example: Say we have a superclass called 'Movie'. Say we have two subclasses of 'Movie' called 'Comedy' and 'Action'. Since it is possible for a movie to have both action and comedy, the two groups are overlapping.

Overlapping Groups

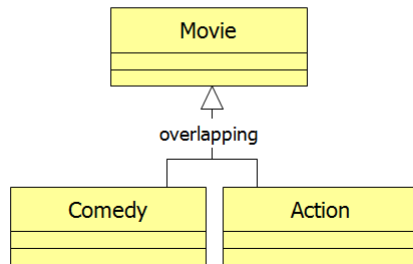


Figure 7.7: Overlapping Groups

Overlapping/Disjoint: Example: Say we have a superclass called 'Book'. Say we have two subclasses of 'Book' called 'Paperback' and 'Hardcover'. Since a book is either a paperback or a hardcover book, the two groups are disjoint.

Disjoint Groups

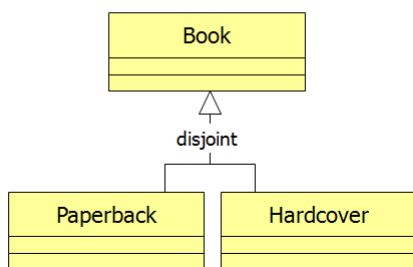


Figure 7.8: Disjoint Groups

Complete/Incomplete

- Complete: The groups listed contain all possible objects that belong to the superclass' group i.e. No more disjoint subclasses can be defined.
- Incomplete: The groups listed do not contain all possible objects that belong to the superclass' group.
- Example: Consider our superclass 'Book' with its subclasses 'Hardcover' and 'Paperback'.
- Are there any other kinds of book?

- The answer is ‘no’, this the groups are complete.

Complete Groups

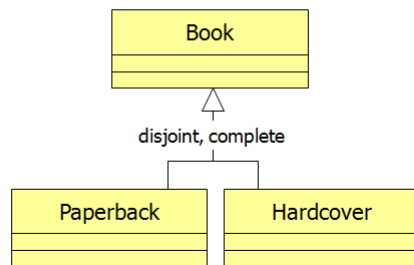


Figure 7.9: Complete Groups

Complete/Incomplete: Example: Consider our superclass ‘Movie’ with its sub-classes ‘Comedy’ and ‘Action’.

- Are there any other kinds of movie?
- The answer is ‘yes’, this the groups are incomplete.

Incomplete Groups

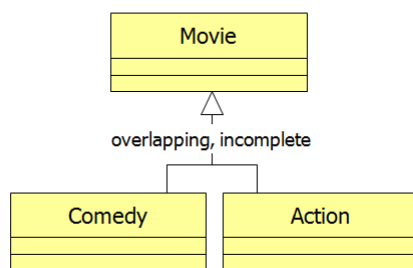


Figure 7.10: Incomplete Groups

7.3.3. Class Diagram: Associations

Relationship where no object is subordinate to others. It is a link between instances (objects) of classes. Associations between classes represent logical relationships objects of those classes might hold.

For example, a ‘Person’ class might have an association ‘HomeAddress’ with another class ‘Location’. This is because an instance of Person might have an instance of Location as its home address. Keep in mind, associates deal with objects, rather

than classes. Thus they declare what associations are possible between their instances. There may be several instances of the association at any time. The number of instances changes over time. One instance may hold several instances of the same association.

An association contains:

- The name of the association
 - This name describes the association
- Multiplicity at each end of the association
 - This describes the number of instances that may occur at that end of the association
- The role of the class at each end (optional)
 - This is usually a name used to describe the instances with respect to that association

Let's illustrate these concepts with an example: Say we have two classes, 'Person' and 'MusicalInstrument', and an association called 'Plays'. The association Plays represents all instances where a Person plays a Musical Instrument.

Association



Figure 7.10: Association Example

- We might have the following instances of 'Plays':
 - Bob plays Piano.
 - Lucy plays Saxophone.
 - Lucy plays Trumpet.
 - Sue plays Drums.
- The multiplicity of Person means:

- How many people can play a given instrument? What is the answer?
 - Any number of people can play a given instrument, including zero. This is represented using 0..*
 - The multiplicity of MusicalInstrument means: How many instruments can a person play?
 - What is the answer? A person may play zero instruments, or any number of them. Again, this is 0..*
- Essentially, an association could be considered a class itself. Every instrument any person plays is an instance of the Plays association.
 - Associations would have members such as 'name', 'multiplicity1', 'multiplicity2', etc..

Association

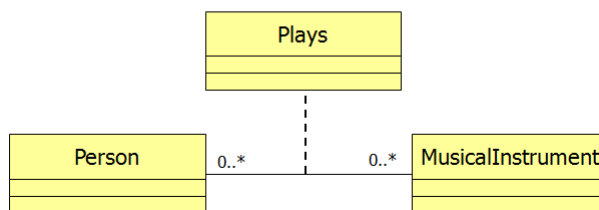


Figure 7.11: Another Association Example

Associations need not always be between 2 classes: Associations can exist between 3 or more classes as well.

7.3.4. Class Diagram: Composition

Composition is a relationship between two classes where one class is a part of another (an 'is a part of' relationship): If a class A is composed of classes B, C, and D, we may say that A has components B, C, and D. Compositions typically consist of a name, as well as the multiplicity of the component. If a class 'Car' has a composition relationship with another class 'Wheel', we should agree that normally the multiplicity of Wheel is 4 (exactly). 'Vehicle', on the other hand, could have any number of wheels (0..*). Composition is a special form of association. It is important enough to have its own notation.

Composition

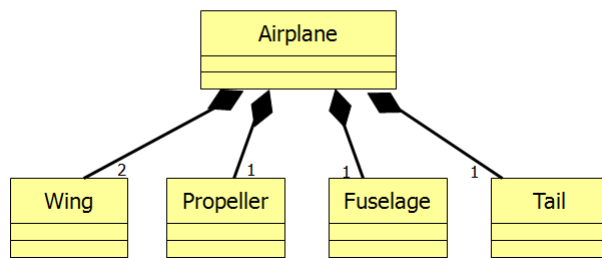


Figure 7.12: Composition Example

The black diamond represents composition. It is placed on the Airplane class because it is the Airplane that is composed of a Wing, Propeller, Fuselage and Tail.

7.3.5. Class Diagram: Aggregation

An aggregation is also a type of association, but again, a special one that is given its own notation. This is because, like composition, it is very common. An association that models a whole-part relationship between an aggregate, which is called whole, and its parts. It is a strong form of association in which the aggregate has no meaning without parts, hence aggregation is known as “part-of” relationship e.g computer made up of parts. An aggregation is another relationship between classes which says one class ‘is a part of’ another class.

Both aggregation and composition represent the ‘is a part of’ relationship. The main difference is what the part represents. In aggregation, the part (constituent) is meaningful without the whole (aggregate). In composition, the part (component) is not meaningful without the whole (container). A side-effect of this difference is that a component is always a part of (at most) one container. However, a constituent may be a member of multiple aggregates.

In UML, composition is considered a special case of aggregation where constituents belong only to a single aggregate. This is why some tools (such as Rational Rose) only have symbols for aggregation, and not composition.

To illustrate a constituent, again, let’s use a few examples:

- A Forest is an aggregate of Trees. o
- A Program is an aggregate of Statements.
- A Fleet is an aggregate of Ships.

- A Deck is an aggregate of Cards.

Parts of an aggregate are called constituents:

- A Tree is an constituent of Forest.
- A Statement is an constituent of Program.
- A Ship is an constituent of Fleet.
- A Card is an constituent of Deck.

These constituents share one common property: The constituents of an aggregate are usually the same type. Two other properties are also defined for aggregation:

- An object may be a constituent of more than one aggregate simultaneously. It is possible to have an aggregate without any constituents
- An object may be a constituent of more than one aggregate simultaneously. This is identified by the multiplicity of the aggregate end of the association.

It is possible to have an aggregate without any constituents. This is identified by the multiplicity of the constituent end of the association.

The weak form of aggregation is denoted with an open diamond. This relationship denotes that the aggregate class (the class with the white diamond touching it) is in some way the “whole”, and the other class in the relationship is somehow “part” of that whole.



Figure 7.13: Aggregation Example

Aggregates can be ordered or unordered. In ordered aggregates, the order of the constituents within the aggregate is important. In unordered aggregates, the order is unimportant. Ordered aggregates are indicated using the symbol [ordered], while unordered is the default for aggregates.

Aggregate Order

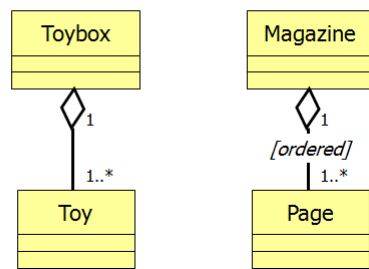


Figure 7.14: Aggregate Order

In programming, aggregation and composition are usually represented by class attributes. However, for clarity, in class diagrams, aggregation and composition should always be used instead of the attribute form. This is because it allows the definition of the class being used to be shown. Say we are writing a word processor: A word processing document consists of a number of paragraphs. Each paragraph consists of a number of elements. An element can be a character or an image.

Class Diagrams: An Example

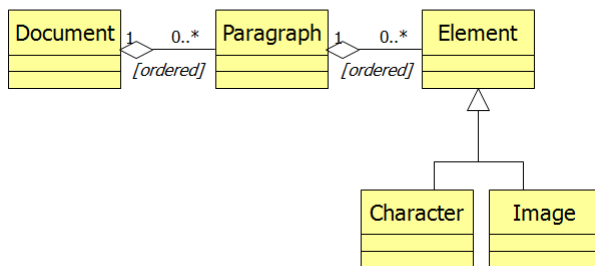


Figure 7.15: Class Diagram

7.4. Example

Every student at JKUAT can be a member of none or many clubs. Each club must have at least one student as a member, besides the club officials. Each club must have at least three club officials and just one chairperson. Each club official can be an official to any other club(s). Draw a class diagram showing at least one attribute for each class.

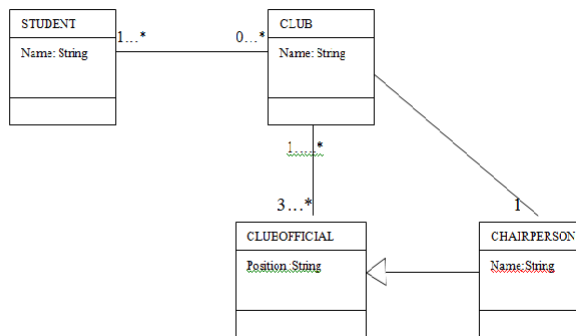


Figure 7.15: Club Class Diagram

7.5. Class Stereotypes

Stereotypes are the core extension mechanism of the UML. If you find that you need a modeling construct that isn't in the UML but is similar to something that is, you treat your construct as a stereotype of the UML construct. A stereotype name is surrounded by Gullimets symbols << >>

An analysis class can be classified into one of the following three stereotypes:

1. Entity classes: Used to model information and associated behavior of some phenomenon or concept such as an individual, a real-life object, or a real-life event.
2. Boundary classes: Used to model interaction between the system and its actors.
3. Control classes: Used to represent coordination, sequencing, transactions and control of other objects.

Consider the class diagram for Add a New Advert to a campaign.

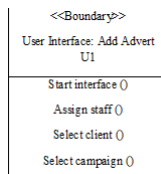


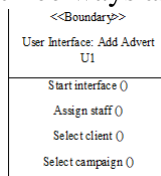
Figure 7.16: Class Stereotypes

7.5.1. Boundary

This is a stereotype class that provides an interface between users and other systems. It models interactions between the system and its actors.

Stereotypes are relatively abstract i.e. they do not directly represent all sorts of interfaces such as buttons, windows, menus, list boxes, e.t.c. that appears on the screen that will be used in the implementation language.

With these stereotypes, we are only interested with identifying the main logical interface with other software and also with physical devices such as monitors, sensors, printers, e.t.c. E.g. the class user interface: Add Advert U1 shown in the collaboration and class diagram is a typical boundary class that can be illustrated in three ways as follows:



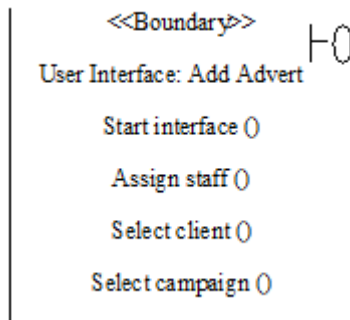
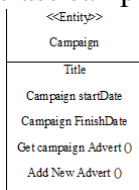


Figure 7.17: Class Boundary

7.5.2. Entity Classes

This is a stereotype class that represents objects in the business domain model. They are used to model information and associated behavior of some phenomenon or concept, such as an individual, a real life object or a real life event as a general rule. Entity classes represent something within the application domain but external to software system about which the system must store information. E.g. an entity class campaign can be shown in three ways as follows:



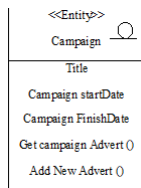
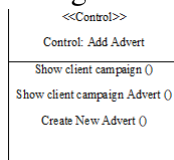


Figure 7.17: Entity Class

7.5.3. Control Classes

This is a stereotype class that controls the interaction between boundary classes and entity classes. It represents coordination sequence transactions and control of other objects. In UML it is generally recommended that there should be a control class for each use case. Control class represents a calculation and scheduling aspects of the logic of the use case. The control class can be shown in three ways as follows.



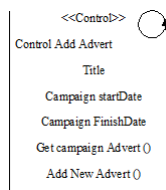


Figure 7.18: Control Class

7.6. Summary

In class diagrams, there are three basic types of classes:

1. Entity classes: Used to model information and associated behavior of some phenomenon or concept such as an individual, a real-life object, or a real-life event.
2. Boundary classes: Used to model interaction between the system and its actors.
3. Control classes: Used to represent coordination, sequencing, transactions and control of other objects.

Learning Activities

Use the following Description of the Classes to draw a Class Diagram for the class timetabling system.

The classes defined for the JKUAT class timetabling system are: User, Primary User, Timetabling Master, Maintainers (IT & Software developer), Login GUI, Security System, Database Manager (DB), Timetabling System, User Details DB, Timetabling Database, Printer and Disk.

The timetabling system shall be connected to two main databases: the database with user login details and the database with timetabling details such as courses information, room information and so on. The UserDetailsDB class connects to the User details database. The timetableDB class connects to the database with timetabling details.

The DBmanager class opens a connection to the database server, sends a query and closes the connection. The Timetabling system class gets users queries, and gets the result from the timetable database. This class also accepts data to be input and modified, which is subsequently updated to the database.

The User class represent a general user of the system, a user can have the role of primary user (student, instructor, other university staff), timetabling master or IT department. This class will store all common data and functions of users. The login function will call the Security class to validate the user's details. The logout functions will simply logout the user.

The Login GUI class represents the login GUI window which is opened by the user. Here, the user simply enters their login details and if log in details are wrong, gets the error message. The user can log out from this window.

The Security System class provides user validation which checks user details from the database. When user is validated, Userwindow is called to display the relevant window for the user depending on their security level, otherwise ErrorLoginMsg is called. Many users can be validated by one security system. Once a user is validated, the useWindow class displays the relevant window for users, depending on their security levels. A user can exit from this window.

The primary user class provides the user with possibility to query the timetable, print or save queried data.

Revision Questions

Example . What is the difference between an aggregation and an association?

Solution: for revision



EXERCISE 13.  Describe the FIVE steps involved in developing a class diagram from a problem description

EXERCISE 14.  With the help of a diagram explain the characteristics of a class

EXERCISE 15.  Draw a class diagram to represent a company that meets the following description: ABC corporation has several employees. For each employee the company keeps track of that employee's last name, first name, middle name, gender, home address, date hired and birth date. Some employees are salaried employees. For these employees, the company is interested in their annual salary. Other employees are hourly employees, the company is interested in knowing their hourly rate.

References and Additional Reading Materials

1. Richard Lee , William M.Tepfenhart (2000). UML and C++ - A practical guide to object oriented development” , ISBN-13: 978-0130290403
2. John. W Satzinger (2004). Object Oriented Analysis and Design with the Unified Process. Cengage Learning, ISBN-13: 978-0619216436
3. Journal of computer science and Technology ISSN 1000-9000

LESSON 8

Interaction Diagrams

Learning outcomes

Upon completing this topic, you should be able to:

1. Describe the different interaction diagrams
2. Model the requirements from the use case diagram to all of the models examined.

8.1. Interaction Diagrams

There are two types of interaction diagrams namely:

1. Collaboration diagrams
2. Sequence diagrams

Both of which express similar information but in different ways. The sequence diagram shows the explicit sequencing of messages while collaboration diagrams shows the relationship among objects.

8.2. Sequence Diagram

It can also be defined as a dynamic model that shows the explicit sequence of messages that are passed between objects in a defined interaction because the pattern of interaction varies from one use case to another.

Collaboration diagrams are used to show simple interactions between objects. While it is possible to define complicated behaviour using collaborations, often the diagrams become very visually complicated and difficult to follow. Sequence diagrams are specifically suited to defining complex interactions over a period of time. It is important to remind you that collaboration and sequence diagrams can be used interchangeably.

However, each have their specific uses. Sequence diagrams show interactions between objects over time. Sequence diagrams are often preferable to collaboration diagrams when:

- There are several interactions involved in a particular behaviour. The sequence (temporal placement) of these interactions gets fairly complicated

Consider an operation called purchase in a warehouse environment. It may involve a number of steps:

- Charge the price of the item (plus tax) to the customer's account
- Removing the product from the inventory database
- Creating a shipping order for the product when it arrives so the item will be delivered to the customer's address

Sequence Diagrams: Example

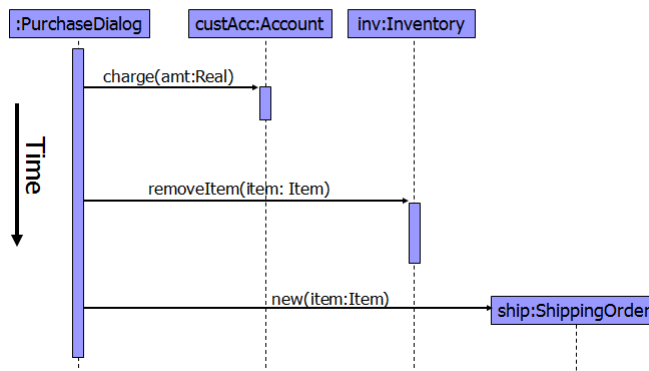


Figure 8.1: Sequence Diagram

Sequence diagrams capture the behavior of single use cases. Hence a sequence diagram shows how objects may interact to realize a use case; each sequence diagram specifies the main interaction steps to be achieved for each use case. Use-cases are broken down into sequences of actions and events. These actions and events are mapped onto objects through object interaction diagrams.

8.2.1. Elements of a Sequence Diagram (Notations/Symbols)

1. An actor

This is a person or system that derives benefits from the system and is external to this system, who participate in a sequence by sending or receiving messages. It is placed across the top of a diagram.

1. An Object/Class:

This is an object/class or object instances that participate in sequence by sending of/or receiving messages. Actor's classes, objects or object instances are shown at the top of the diagram. The left most object is the starting object and maybe a person (for which a use case actor symbol is used), window or dialog box or other user interfaces. The top messages use indicators in their name to indicate who the rectangle represents. This can be shown as follows.

Object Name

: Class

Object:: Class

1. Life line

This denotes the life of an object that sequence. It presents the life for the class of an object flow when it was created through and when it is destroyed.

1. Focus of control

This shows/represents an activation of the object of class. Activation shows the time period during which the object or class performs an operation, either directly or through a call to some subordinate operation. The bar is placed at the top of a lifeline. It shows when an object is busy doing things i.e sending or receiving messages.

1. A message

This is the interaction from one object or class to another. A message conveys information from object to another one. Messages belong to the receiving class or objects. There are variations in message arrows. These are:

- Solid Arrow (Full arrow)

This represents synchronous messages. A type of message in which the caller has to wait for the receiving objects to finish executing the call operation before it can resume execution itself. They are used when the sending class and the control is returned to the sending class when the receiving message finishes the execution. Synchronous messages always have associated return messages. The message only may provide the caller with some return value(s) or simply back to the caller that the operation called has been successfully completed.

- Open Arrow (Half arrow)

This represents asynchronous messages. This is a message in which the sender does not have to wait for the recipient to handle the message. The sender can continue executing the message of immediately after sending the message. Asynchronous messages are common in concurrent real time system where several objects operate in parallel.

- Dashed Full Arrow

Used to show a returning message.

- Object Destruction

Used to show when an object goes out of existence. It represents when the object is destroyed. Placed at the end of an object's lifeline.

8.2.2. Message Labeling

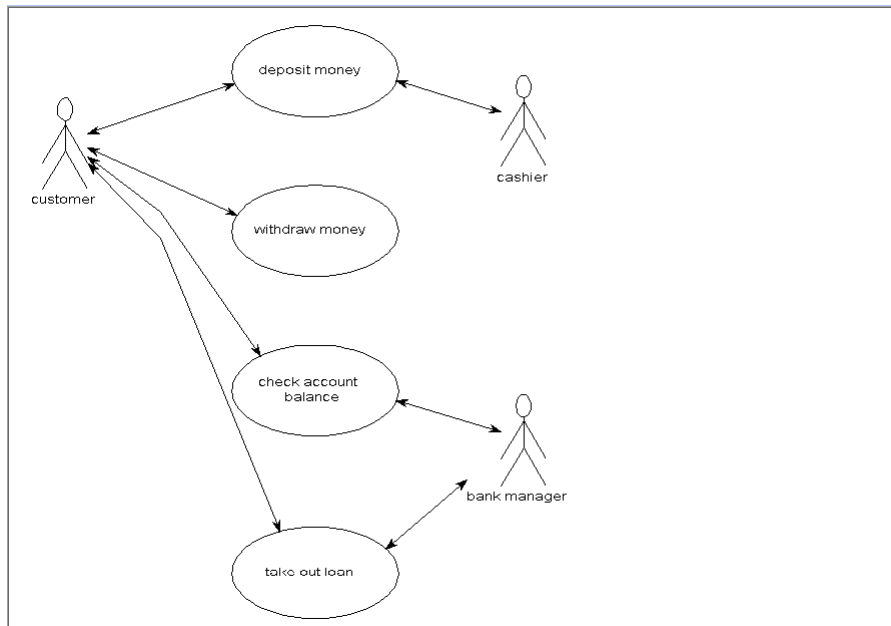
Messages can be labeled using the following:

- The name of message followed by an empty parenthesis e.g. message name ()
- The name of message followed by parameters in the parenthesis e.g. message name (parameter1, ———etc)
- The name of the message followed by parameter type, parameter name and any default value for the parameter in the parenthesis e.g. message name (parameter type, parameter name, default value).
- The message may be a stereotype such as <create> indicating that a new object is created as a result of some actions.

NB:

Timing in the sequence diagram is displayed from top to bottom. The first interaction is shown at the top of the diagram and the last is showed at the bottom of a diagram. The interaction allows begins at the top of the bar of actor or object.

Sequence diagrams can be used to translate the use case scenario into a visual tool for system analysis. The initial sequence diagram used in system analysis shows the actors and classes in the system and interactions between them for a specific process.

Example: Consider the bank example**Figure 8.2: Bank Sequence Diagram**

The deposit cash involves the following sequence of actions:

1. Complete pay in slip
2. Hand cash to cashier
3. Increase cash balance of account
4. Increase cashier's balance
5. Increase bank's balance
6. Issue receipt to customer

It could be more complicated if we included cheques, but you can do that as an exercise. The link between the requirements model and the object model is built through object interaction diagrams. We can now construct an object interaction diagram something like:

Complete pay-in slip
 Hand pay-in slip to cashier
 Increase customer account balance
 Increase cashier balance
 Increase banks overall balance
 Issue receipt to customer



Figure 8.3: Bank Object Interaction Diagram

Now we have to consider what objects we might need for interaction. The first debate is whether we want to model the pay-in slip. In the final analysis, the decision to model something as an object depends on how important the behaviour is to the system we are developing. It might be that we only need to record the existence of the pay-in slip, and it can be referenced by an attribute. But to be safe, at first we shall include it as an object.

Complete pay-in slip
 Hand pay-in slip to cashier
 Increase customer account balance
 Increase cashier balance
 Increase banks overall balance
 Issue receipt to customer

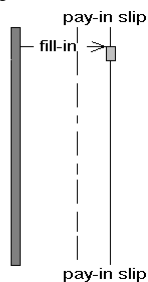


Figure 8.4: Bank Pay-in Slip

Next we have to decide if we wish to model the cashier. Again, at first it is better to model something as an object and reduce its status to an attribute (say) or remove it from the system at a later stage. So we include a second object.

Complete pay-in slip
 Hand pay-in slip to cashier
 Increase customer account balance
 Increase cashier balance
 Increase banks overall balance
 Issue receipt to customer

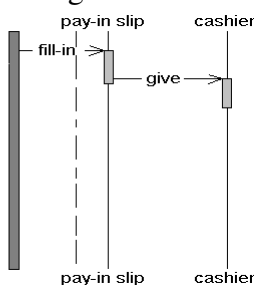


Figure 8.4: Bank Pay-in Slip Cashier Interaction

Now we have to worry about the customer and his/her account(s). Now a customer may have more than one account, normally. So it seems sensible to model both. Here, however, we merely need to know about the account. In fact the person

depositing money may not be the owner of the account.

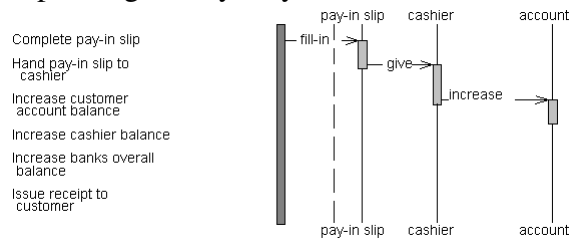


Figure 8.5: Bank Pay-in Slip Cashier Interaction

Now we are left with trying to understand where the cashier balance might be stored. We might want to create an object for this, or store this as an attribute of the cashier object. There seems no need to keep a cashier balance separate, so we shall record the cashier balance in the cashier object.

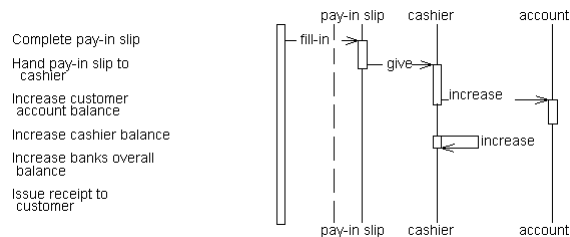


Figure 8.6: Bank Pay-in Slip Cashier Interaction

Now we are left with a decision about whether to keep a bank object. It seems reasonable, even though there may only be one. (Alternatively, we might think of the bank having many branches, and have to increment the branch balance. Let's keep it simple though.)

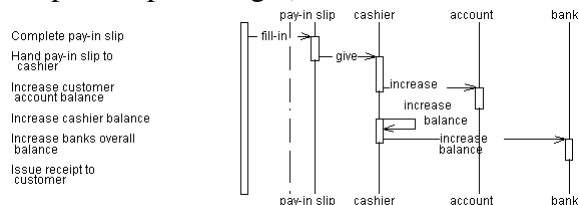


Figure 8.7: Bank Pay-in Slip Cashier Interaction

Now we have to decide whether to include the customer object. At this stage, the thing we are talking about is a real, physical customer. Hence we might leave a customer object out of the interaction diagram.

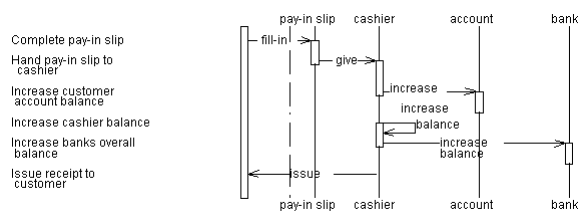


Figure 8.8: Bank Pay-in Slip Cashier Interaction

And so we progress. Now it is almost certain that two analysts working on the system would come up with different decisions at each stage. There is no single, definitive answer. Hopefully the set of objects derived by independent analysts would be similar, but any expectation that analysis is a deterministic activity with right or wrong answers is bound to be frustrated. Of course, there are good and bad analyses and designs.

A large part of what makes for good design comes from experience. In a nutshell, the first step from requirements analysis to detailed analysis is through the use of object-interaction diagrams to define objects, their behaviour and their attributes. These objects will be found in many interaction diagrams, and their overall behaviour is a composite of the behaviours from the individual interaction diagrams. Having found some of the behaviours of objects in this way, we can then produce state-transition diagrams for the objects to more tightly define their behaviour.

Our initial analysis can progress beyond functionality towards some architecture to support that functionality. There are many techniques for driving the analysis. An analysis should involve the appropriate parties in the development of the system. There is not enough scope in this course to discuss methods of interviewing. One useful technique, however is to provide some discussion around the interface of the use-cases with the actors.

This has the advantage that it begins to open up the way the system may be used, and it defines both attributes and operations the use-case needs to take into consideration. For example, suppose we wished to discuss with a bank manager what happened when he/she checked an account balance. We might draw up a possible screen and describe it to the bank manager, asking for feedback on the suitability of the screen, its contents, and what the bank manager would expect to be able to do through the screen. It can be related to the use case diagram as below.

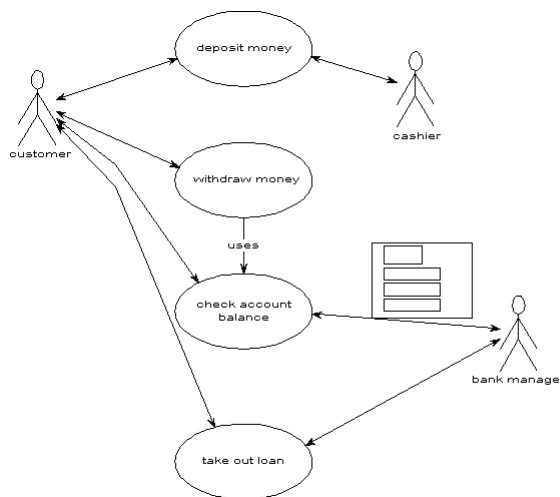


Figure 8.9: Bank Use Cases

As the analysis progresses through to design, interface objects will have to be designed to drive the interfaces with the various actors, and effectively implement the use-cases as objects in their own rights.

Use-cases can be thought of as objects whose behaviour is defined in terms of other objects in the system. Abstract analysis and design can progress a long way before taking into consideration the hardware and software requirements for implementation.

Arguably it is better to defer the decisions about implementation as far down the line as possible, though in practice it is often important to have some idea of the basic hardware and software configurations which will be available; in many practical projects these will be given as part of the existing infrastructure or policy of the client.

In developing sequence diagrams and consequently distributing the responsibilities the following guidelines need to be considered:

1. Employ the use case diagram to determine the use cases. Determine the actors that interact with the use case. Don't model interactions between actors since they are outside the scope of the system
2. Consider one use case at a time. Obtain the text-based , detailed description of each use case
3. For a given use case, specify a boundary class between the use case and each actor. Thus, if there are two actors, there will be two boundary classes

4. For a given use case, specify one control class
5. Derive the entity classes from object relations for the use case
6. Interactions in a sequence diagram are among objects of the classes, not among the classes themselves. Use the notation :Customer to indicate an object of the class Customer

8.3. Summary

Learning Activities

1. Now draw sequence diagrams for the class timetabling system. For example, the one below is the sequence diagram for LOGIN.

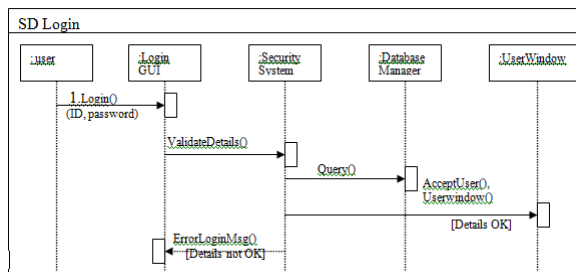


Figure 8.10: Login Sequence Diagram

8.4. Collaboration Diagram

Structured from the perspective of interactions among objects. It captures dynamic behavior (Message-oriented).

Its purposes is to:

- Model flow of control
- Illustrate coordination of object structure and control
- Objects that interact with other objects
- Sequence::Time vs. Collaboration::Message

Main Concepts: Collaboration, Interaction, Collaboration Role, Message.

Collaboration diagram captures dynamic behavior (Message-Oriented)

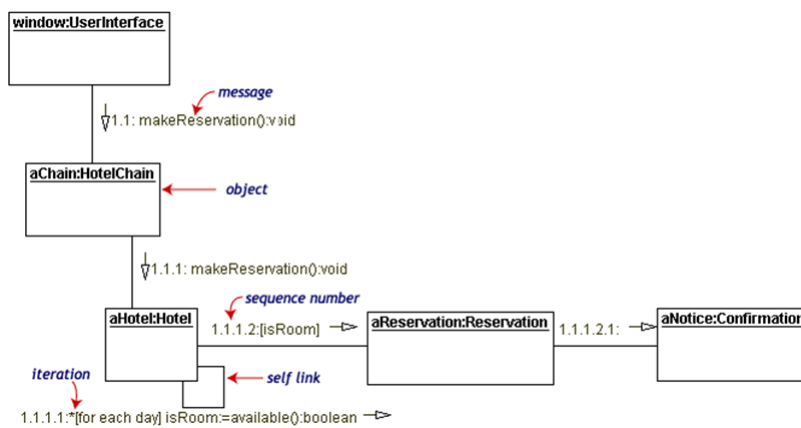


Figure 8.11: Collaboration Diagram

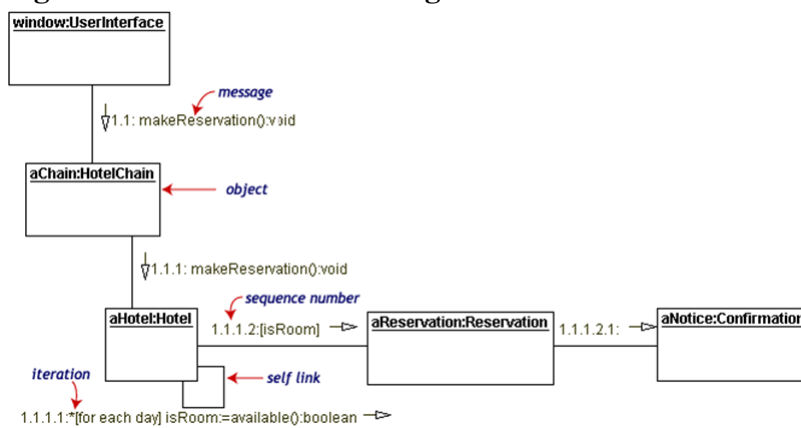


Figure 8.12: Collaboration Diagram

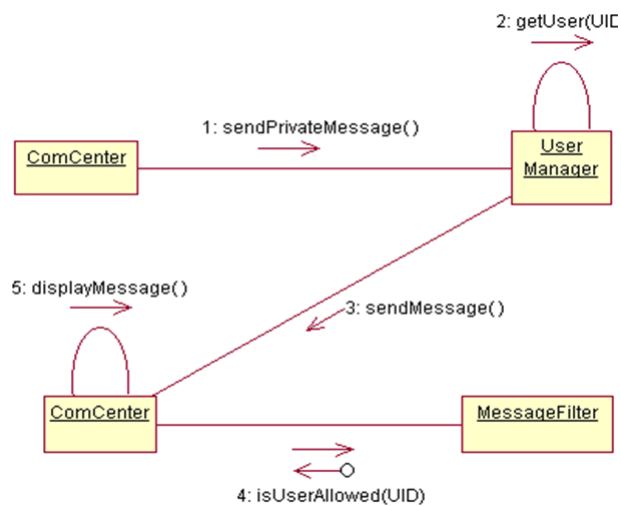


Figure 8.13: Collaboration Diagram

Learning Activities

Explain how use cases and class diagrams are related to interaction diagrams.

Revision Questions

Example . Explain the difference between a state diagram and an activity diagram

Solution: for revision



EXERCISE 16.  Explain the elements that make up an activity diagram

EXERCISE 17.  Create a state diagram for the following scenario concerning a Car class A car can be booked via an order, if the date is free. A Car can be either in Store or on Loan depending on the bookings. A car can be booked for a future date whether it is on loan or not.

References and Additional Reading Materials

1. Richard Lee , William M.Tepfenhart (2000). UML and C++ - A practical guide to object oriented development” , ISBN-13: 978-0130290403
2. John. W Satzinger (2004). Object Oriented Analysis and Design with the Unified Process. Cengage Learning, ISBN-13: 978-0619216436
3. Annals Of software Engineering ISSN 1022-7091

LESSON 9

Other UML Diagrams

Learning outcomes

Upon completing this topic, you should be able to:

1. Describe state transition, activity and implementation diagrams
2. Model the requirements from the use case diagram to all of the models examined.

9.1. Development of Dynamic Behavior

Use cases suggest the system events which are explicitly shown in system interaction diagrams. An initial, textual description of the system events and their goal is presented by operation contracts. The system events represent messages that initiate interaction diagrams, which illustrate how objects interact to fulfill the required tasks.

9.1.1. Dynamic Behavior: Perspective

The dynamic behavior of a use case viewed from the conceptual perspective shows:

- actors of a use case,
- interactions of the actors with the system

The implementation perspective of a use case shows

- the requests from outside (i.e., system operations),
- the internal messages and system events,
- the system as structured components (e.g., classes) involved in the process.

9.1.2. Use Cases and Interaction Diagrams

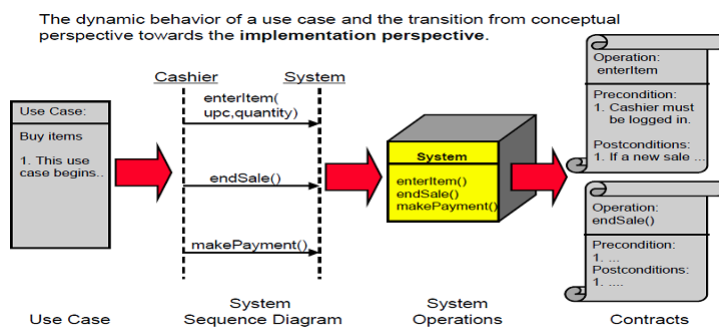


Figure 9.1: Use Cases and Interaction Diagrams

9.1.3. Use Case and System Interaction

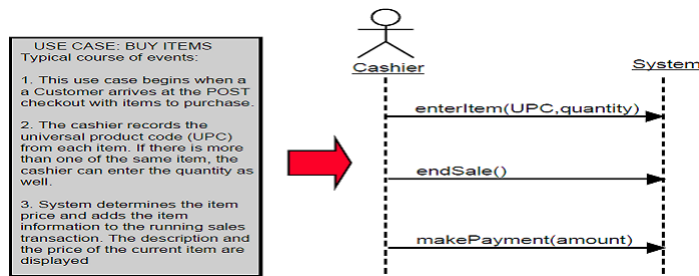


Figure 9.2: Use Cases and Interaction Diagrams

9.1.4. Contracts

The system sequence diagram shows the system events that an external actor generates. It does not elaborate on the details of the functionality associated with the system operations invoked. In general a contract is a document that describes what an operation commits to achieve. A system operation contract describes changes in the state of the overall system when a system operation is invoked.

9.1.5. Contracts and Collaboration Diagrams

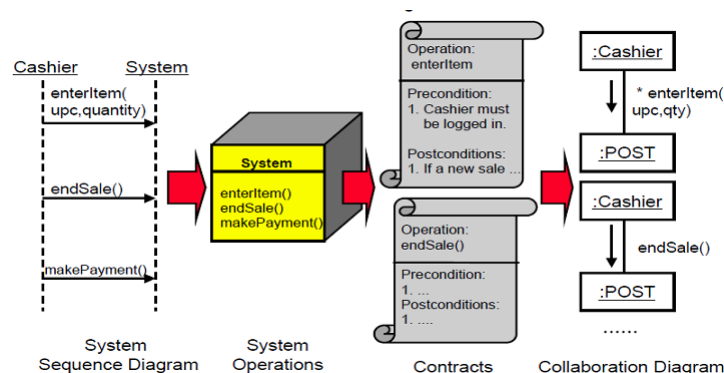


Figure 9.3: Contracts and Collaboration Diagrams

9.2. State Transition Diagram

Sequence and collaboration diagrams are ways of representing all the iterations that take place between objects involved in a particular use case. Sometimes, however, we need to focus on specific class to illustrate how its objects behave during their lifetimes and how they react to all use cases in which they are involved. In this case, the suitable diagram is called the state diagram.

Definition:

A state diagram can also be defined as a model of the state of an object and the events that cause the object to change from one state to another. A state diagram describes the behavior of a single class. It describes how a class of objects behave in the system. In other words, it describes how a class responds to all use cases that affect it.

If a class always responds exactly in the same way to events then there's no point of drawing a state diagram for it. The interest is in classes where the response of an object of the class to a particular event varies depending on the state the object is in at the time. So when you want to find out what's going on within a class, you need a state diagram.

A state transition diagram shows the states of a single object, the events or the messages that cause a transition from one state to another and the action that result from a state change. A state transition diagram will not be created for every class in the system.

9.2.1. Components of State Diagram

1. Start State
2. Stop state
3. State Transition

9.2.2. Semantics of Every Component

1. State:
 - A state is a condition during the life of an object when it satisfies some condition, performs some action, or waits for an event.
 - It represents a period of time during which the object satisfies some conditions for some events.
 - The UML notation for a state is a rectangle with rounded corners.
1. Special states: There are two special states.

- Start state: Each state diagram must have one and only one start state. Notation for start state is “filled solid circle”.
- Stop State: An object can have multiple stop states. Notation for stop state is bull’s eye.

2. Event

- This is something that takes place at a certain period in time.
- It is an occurrence that triggers a state transition.
- Some examples of events are: a customer places some order, a student registers for a class, a person applies for a loan, a company hires a new employee.

1. State transition:

- A state transition represents a change from an originating to a successor state.
- This represents the response of an object to an event.
- The response may involve movement of the object from one state to another or the object remaining the same state (sometimes referred to as self transition).
- The state transition changes in the attributes of an object or in the links and objects as with other objects. The state of an object depends in its attribute values and links to other objects.
- An object remains in a particular state for sometime before transitioning to another state
 - For example an employee object might be in the part time state (as specified in the employee status attribute) for a few months before transitioning to a full time state based on a recommendation from the manager (event).
- A state transition is regarded as instantaneous and cannot be interrupted.
- It consists of three parts: the event, guard and action. All these three parts are optional

- An event triggers a state transition.
- A guard is a condition that allows the transition to take place only if the condition is true. Guards are written inside square brackets [].
- An action is the behavior that occurs when a transition takes place and is presided by a slash (/).
- Both guards and actions are generally implemented as operations on the class in the final system code.
- Transition label: event name[guard condition] / action

9.2.3. Notations

1. Initial state: A filled circle represents the object initial state.
2. Final state: A filled circle nested inside another circle. A bull's eye represents the object final state.
3. State: A rectangle with round corners represents a situation during the life of an object.
4. Transition arrow: This represents the path between different states of an object. The transition is labeled with the event that triggered it and the action that results from it.

Note:

Transition events are named in the past tense because they have already occurred within the transition. State chart diagrams are not created for all classes. They are created when:

1. A full class has a complex lifecycle.
2. An instance of class updates its attributes through the lifecycle.
3. A class has an operational life cycle.
4. Two classes depend on each other.
5. The object's current behavior depends on what happened previously.

9.2.4. Steps for Building State Chart Diagram

1. Set the context: the context of a state chart diagram is usually a class. However, it also could be a set of classes, a subsystem, or an entire system.
2. Identify initial, final and stable states of the object. This involves identifying the various states that an object will have over its lifetime. This includes establishing the boundaries of the existence of an object, by identifying the initial and final states of an object. We must also identify the stable state of an object. The information necessary to perform this step is reading from the use case descriptions, talking with users and relying on the requirement gathering technique. An easy way to identify the states of an object is to write the steps of what happens to an object over time from start to finish.
3. Determine the order in which the object will pass through the stable state. Here we determine the sequence of state that an object will pass through during this lifetime. Using this sequence, we can place the states on the state diagrams in a left to right order.
4. Identify the events, actions and guard conditions associated with the transitions. Events actions and guard conditions associated with the transition between the states of the object are identified and shown on diagram. The events are the triggers that cause an object to move from one state to the next state. In other words, an event causes an action to execute that changes the value (s) of an object attribute (s) in a significant manner. The actions are typically operations contained within the object. Guard conditions can model a set of test condition that must be met for the transition to occur. At this point in the process, the transition are drawn between the relevant states and labeled with the event, action or guard condition.
5. Validate the state diagram. The state diagram is validated by making sure that each state is reachable and that is possible to leave all state except for final state. If an identified state is not reachable either a transition is missing or the state was identified in error. Furthermore, only final state can be a dead end from the perspective of an object lifecycle.

9.2.5. State Transition Diagram for Account Class

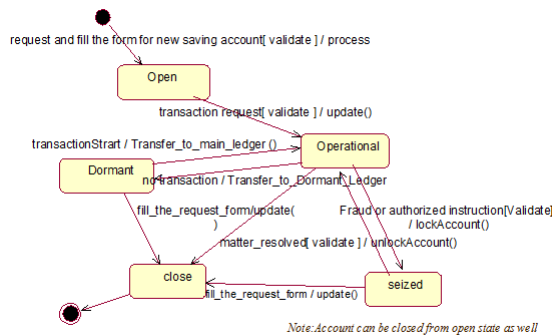


Figure 9.4: State Transition Diagram for Account Class

Note:

- A state diagram will not be created for every class.
- State diagrams are used only for those classes that exhibit interesting behavior.
- State diagrams are also useful to investigate the behavior of user interface and control classes.
- State diagram are used to show dynamics of a individual class

9.3. Activity Diagrams

It is a special kind of state diagram and is worked out at use case level. These are mainly targeted towards representing internal behavior of a use case. These may be thought as a kind of flowchart. Flowcharts are normally limited to sequential process; activity diagrams can handle parallel process.

Activity diagrams are recommended in the following situations:

- Analyzing use case
- Dealing with multithreaded application
- Understanding workflow across many use cases.

They represent the performance of operations and transitions that are triggered. They captures dynamic behavior (Activity-Oriented).

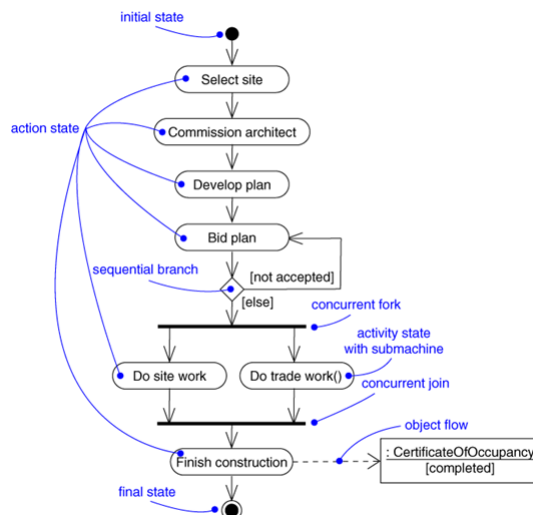


Figure 9.5: Activity Diagram

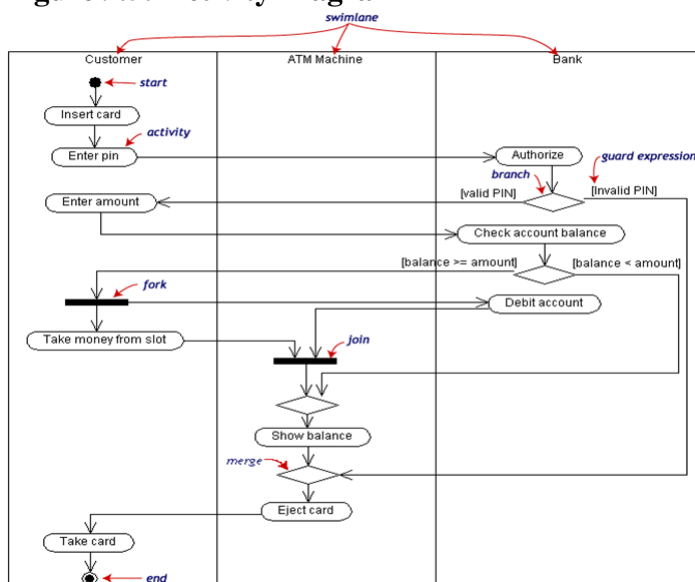


Figure 9.6: Activity Diagram

9.3.1. Consistency Checking

Consistency checking is the process of ensuring that, information in both static view of the system (class diagram) and the dynamic view of the system (sequence and collaboration diagram) is telling the same story.

9.4. Implementation Diagrams

Implementation diagrams show aspects of implementation, including source code structure and run-time implementation structure.

They come in two forms:

1. Component diagrams show the structure of the code itself and
2. Deployment diagrams show the structure of the run-time system.

9.5. Component Diagram

Component diagrams illustrate the organizations and dependencies among software components. A component diagram shows the dependencies among software components, including source code components, binary code components, and executable components.

A software module may be represented as a component type. Some components exist at compile time, some exist at link time, some exist at run time, and some exist at more than one time. A compile-only component is one that is only meaningful at compile time. The run-time component in this case would be an executable program. A component diagram has only a type form, not an instance form. To show component instances, use a deployment diagram (possibly a degenerate one without nodes). A component may be

- A source code component
- A run time components
- An executable component
- Dependency relationship.

9.5.1. Notation

A component diagram is a graph of components connected by dependency relationships. Components may also be connected to components by physical containment representing composition relationships. A diagram containing component types and node types may be used to show compiler dependencies, which are shown as dashed arrows (dependencies) from a client component to a supplier component that it depends on in some way.

The kinds of dependencies are language-specific and may be shown as stereotypes of the dependencies. The diagram may also be used to show interfaces and calling dependencies among components, using dashed arrows from components to interfaces on other components. Has only a type form, not an instance form.

To show component instances, use a deployment diagram (possibly a degenerate one without nodes).

9.5.2. Component Diagram for Withdrawal of Cash

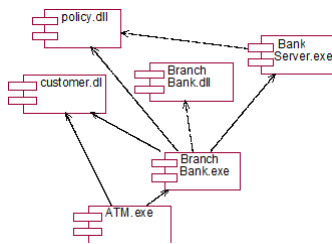


Figure 9.7: Cash Withdrawal Component Diagram

9.6. Deployment Diagram

Deployment diagrams show the configuration of run-time processing elements and the software components, processes, and objects that live on them. Software component instances represent run-time manifestations of code units. Components that do not exist as run-time entities (because they have been compiled away) do not appear on these diagrams, they should be shown on component diagrams. A deployment diagram shows the relationship among software and hardware components in the delivered system. These diagrams include nodes and connections between nodes. Each node in deployment diagram represents some kind of computational unit, in most cases a piece of hardware. Connection among nodes show the communication path over which the system will interact. The connections may represent direct hardware coupling line RS-232 cable, Ethernet connection, they also may represent indirect coupling such as satellite to ground communication.

9.6.1. Notation

A deployment diagram is a graph of nodes connected by communication associations. Nodes may contain component instances. This indicates that the component lives or runs on the node. Components may contain objects, this indicates that the object is part of the component. Components are connected to other components by

dashed-arrow dependencies (possibly through interfaces). This indicates that one component uses the services of another component. A stereotype may be used to indicate the precise dependency, if needed. The deployment type diagram may also be used to show which components may run on which nodes, by using dashed arrows with the stereotype «supports». Migration of components from node to node or objects from component to component may be shown using the «becomes» stereotype of the dependency relationship. In this case the component or object is resident on its node or component only part of the entire time. Note that a process is just a special kind of object.

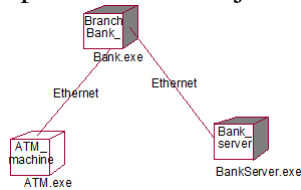


Figure 9.8: Deployment Diagram

9.7. Package Diagrams

Package diagrams are a valuable tool for grouping logically depending elements together.

For packaging, place those objects together which:

- are in the same subject area - closely related by concept or purpose,
- are in a type hierarchy,
- participate in the same use case,
- are strongly associated.

9.8. Summary

Learning Activities

Develop a simple sequence diagram in which: a) an object O receives message m from an actor; b) O creates a new object P; c) P sends a message to Q; d) P returns after receiving the response from Q; e) O destroys P

Revision Questions

Example . Explain the difference between a state that has no outgoing transition at all, and one with an arrow into a stop (end) marker.

Solution: for revision



EXERCISE 18.  Explain the difference between component diagram and a deployment diagram.

EXERCISE 19.  Describe the purpose of an activity diagram

References and Additional Reading Materials

1. Richard Lee , William M.Tepfenhart (2000). UML and C++ - A practical guide to object oriented development” , ISBN-13: 978-0130290403
2. John. W Satzinger (2004). Object Oriented Analysis and Design with the Unified Process. Cengage Learning, ISBN-13: 978-0619216436
3. Journal of computer science and Technology ISSN 1000-9000

LESSON 10

Production of Specification Document

Learning outcomes

Upon completing this topic, you should be able to:

1. Explain the goals of UML
2. Describe several UML software packages and their capabilities.
3. Translate the specification document into code using the chosen programming language.

10.1. Architecture and the UML

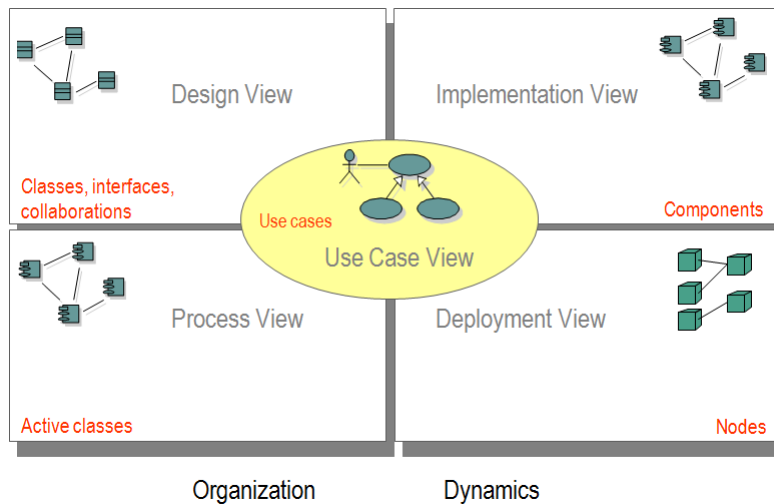


Figure 10.1: Architecture and the UML

10.1.1. Goals of the UML

The primary design goals of the UML are as follows:

1. Provide users with a ready-to-use, expressive visual modeling language to develop and exchange meaningful models.
2. Provide extensibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development processes.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of the OO tools market.
6. Support higher-level development concepts such as collaborations, frameworks, patterns, and components.
7. Integrate best practices.

Provide users with a ready-to-use, expressive visual modeling language to develop and exchange meaningful models

It is important that the OOAD standard supports a modeling language that can be used "out of the box" to do normal general-purpose modeling tasks. If the standard merely provides a meta-meta-description that requires tailoring to a particular set of modeling concepts, then it will not achieve the purpose of allowing users to exchange models without losing information or without imposing excessive work to map their models to a very abstract form.

The UML consolidates a set of core modeling concepts that are generally accepted across many current methods and modeling tools. These concepts are needed in many or most large applications, although not every concept is needed in every part of every application. Specifying a meta-meta-level format for the concepts is not sufficient for model users, because the concepts must be made concrete for real modeling to occur.

If the concepts in different application areas were substantially different, then such an approach might work, but the core concepts needed by most application areas are similar and should be supported directly by the standard without the need for another layer.

Provide extensibility and specialization mechanisms to extend the core concepts

OMG expects that the UML will be tailored as new needs are discovered and for specific domains. At the same time, we do not want to force the common core concepts to be redefined or re-implemented for each tailored area.

Therefore, we believe that the extension mechanisms should support deviations from the common case, rather than being required to implement the core OOA&D concepts themselves. The core concepts should not be changed more than necessary.

Users need to be able to:

- build models using core concepts without using extension mechanisms for most normal applications,
- add new concepts and notations for issues not covered by the core,

- choose among variant interpretations of existing concepts, when there is no clear
- consensus, and
- specialize the concepts, notations, and constraints for particular application domains.

Be independent of particular programming languages and development processes

The UML must and can support all reasonable programming languages. It also must and can support various methods and processes of building models.

The UML can support multiple programming languages and development methods without excessive difficulty.

Provide a formal basis for understanding the modeling language

Because users will use formality to help understand the language, it must be both precise and approachable; a lack of either dimension damages its usefulness. The formalisms must not require excessive levels of indirection or layering, use of low-level mathematical notations distant from the modeling domain, such as set-theoretic notation, or operational definitions that are equivalent to programming an implementation.

The UML provides a formal definition of the static format of the model using a metamodel expressed in UML class diagrams. This is a popular and widely accepted formal approach for specifying the format of a model and directly leads to the implementation of interchange formats. UML expresses well-formedness constraints in precise natural language plus Object Constraint Language expressions.

UML expresses the operational meaning of most constructs in precise natural language. The fully formal approach taken to specify languages such as Algol-68 was not approachable enough for most practical usage.

Encourage the growth of the OO tools market

By enabling vendors to support a standard modeling language used by most users and tools, the industry benefits. While vendors still can add value in their tool

implementations, enabling interoperability is essential. Interoperability requires that models can be exchanged among users and tools without loss of information. This can only occur if the tools agree on the format and meaning of all of the relevant concepts. Using a higher meta-level is no solution unless the mapping to the user-level concepts is included in the standard.

Support higher-level development concepts such as collaborations, frameworks, patterns, and components

Clearly defined semantics of these concepts is essential to reap the full benefit of OO and reuse. Defining these within the holistic context of a modeling language is a unique contribution of the UML.

Integrate best practices

A key motivation behind the development of the UML has been to integrate the best practices in the industry, encompassing widely varying views based on levels of abstraction, domains, architectures, life cycle stages, implementation technologies, etc. The UML is indeed such an integration of best practices.

10.1.2. Object-Oriented Developing Process

UML is only a notation, not a method. We need an object-oriented developing process.

Famous developing process

- Rational Unified Process
- OMT,
- Booch Method,
- XP, ...

10.1.3. Steps Used in UML

The steps used in UML are:

- Define the use case model
- Define the object model

- Continue UML diagramming to model the system during the systems analysis phase
- Begin system design by refining UML diagrams and using them to derive classes and their properties and methods

10.1.4. Difficulties of Object-Oriented Design

Object Oriented Design

- find pertinent objects,
- factor them into classes at the right granularity,
- define class interfaces and inheritance hierarchies, and
- establish key relationships among them

The design should be specific to the problem at hand but also general enough to address future problems and requirements. Also want to avoid redesign, or at least minimize it . A reusable and flexible design is difficult if not impossible to get “right” the first time.

10.2. From UML to the Unified Process

UML as a model can't work in isolation. Large scale system design/development involves:

- Team-Oriented Efforts.
- Software Architectural Design
- System Design, Implementation, Integration

The Unified Process by Rational is:

- Iterative and Incremental
- Use Case Driven
- Architecture-Centric

10.2.1. Creating the Unified Process

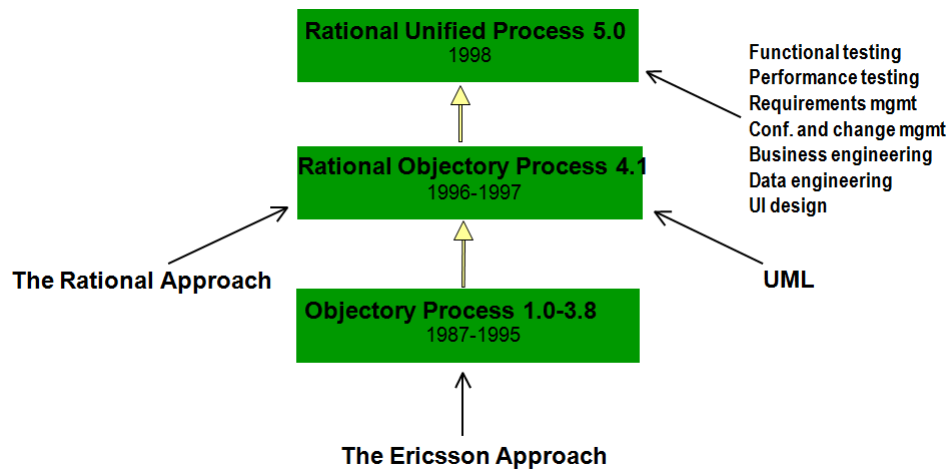


Figure 10.2: Creating the Unified Process

10.2.2. What Is a Process?

It defines who is doing what, when to do it, and how to reach a certain goal.

10.2.3. Lifecycle Phases

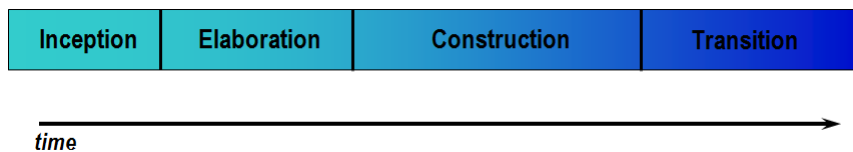


Figure 10.3: Unified Process Phases

- Inception: Define the scope of the project /develop business case
- Elaboration: Plan project, specify features, and baseline the architecture
- Construction: Build the product
- Transition: Transition the product to its users

10.2.4. Unified Process Structure Iterations and Workflow

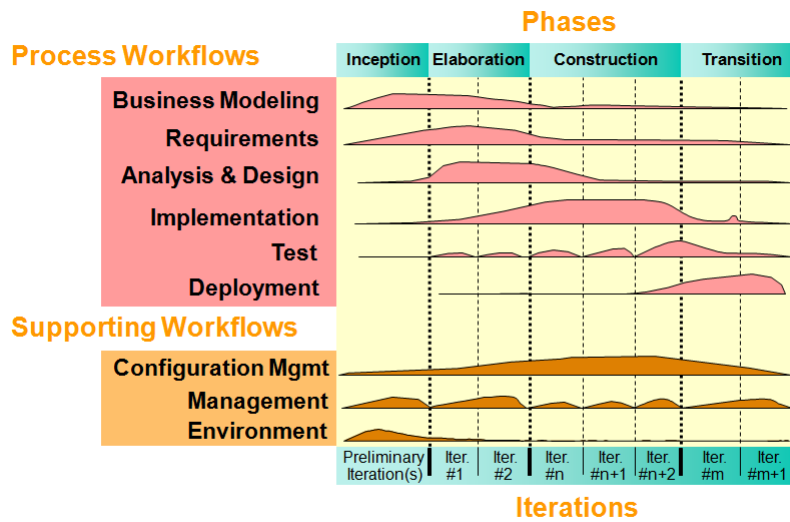


Figure 10.4: Unified Process Workflows

10.2.5. Workflows and Models

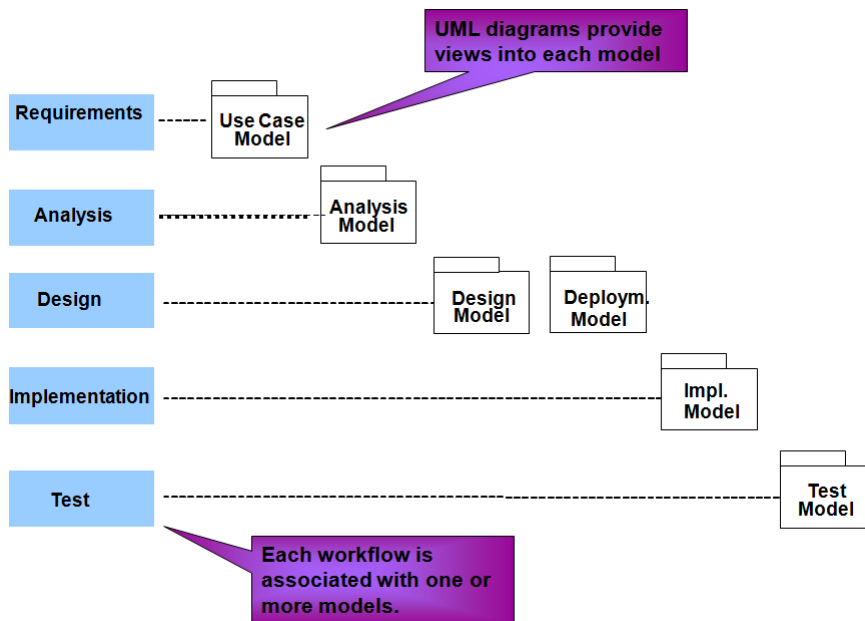


Figure 10.5: Unified Process Workflows and Models

10.2.6. Use Case Model

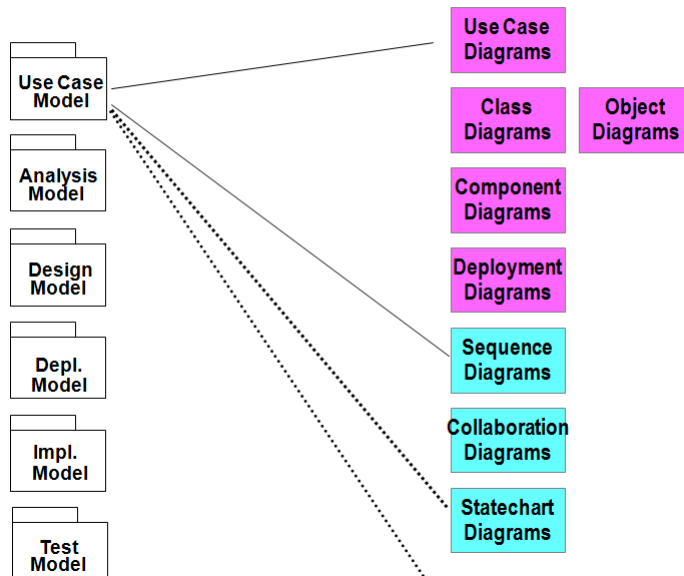


Figure 10.6: Use Case Model

10.2.7. Analysis and Design Model

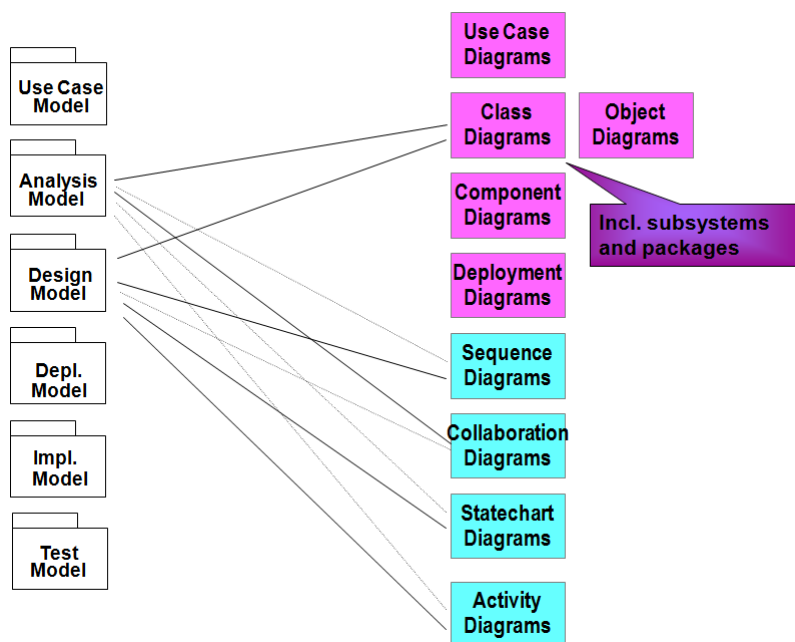


Figure 10.7: Analysis and Design Model

10.2.8. Deployment and Implementation Model

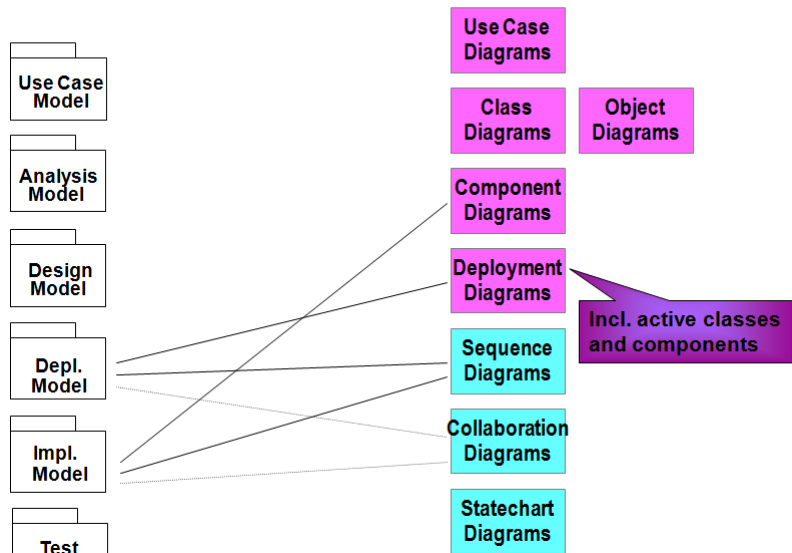


Figure 10.8: Deployment and Implementation Model

10.2.9. Test Model

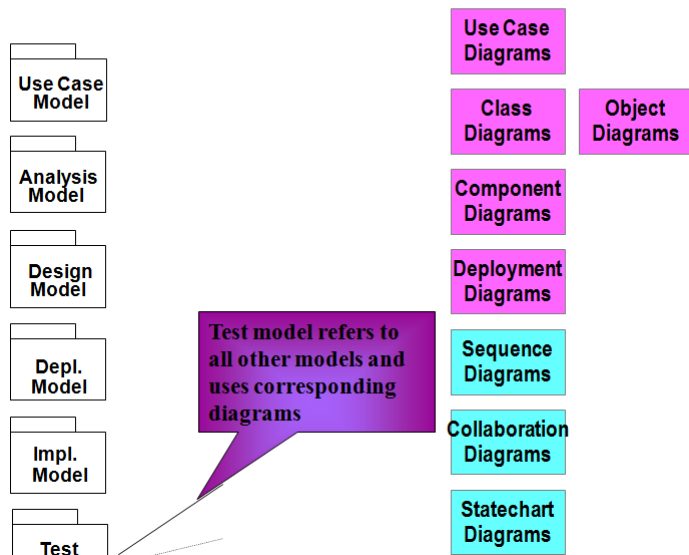


Figure 10.9: Test Model

10.2.10. Use Case Driven

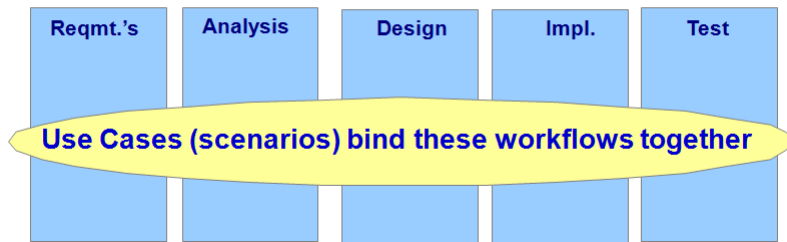


Figure 10.9: Use Case Driven Development

Use Cases Drive Iterations

Drive a number of development activities

- Creation and validation of the system's architecture
- Definition of test cases and procedures
- Planning of iterations
- Creation of user documentation
- Deployment of system

Synchronize the content of different models

10.3. Production of Specification Document

10.3.1. Document Outline

1. Introduction

The document begins with an introductory description of the desired system. Except as noted below, the system is described in present tense, third person, active voice. The purpose of using present tense is to render this document historically relevant throughout the lifetime of the system. That is, the requirements specification is a permanent part of the system documentation, before the system is implemented, during implementation, and afterward. Later, the system is referred to in future tense in order to distinguish between the state of operations before and after system installation.

1. Problem Statement

The problem statement is a succinct presentation of the specific problem(s) to be solved and the needs to be met by the system.

1. System Personnel

The description of system personnel defines all of the people involved with the system and its development. Personnel categories include some or all of the following:

- system end users
- paying customers
- project managers
- domain experts
- system analysts
- system developers

Membership among these groups may overlap.

1. Operational Setting

The operational setting is the environment in which the system is to be used. The discussion includes how operations are conducted in the environment before and after the system is installed.

The following questions are addressed in the discussion of the operational setting:

- What computer-based support is in use prior to installation of the new system?
- Does the new system need to interface with existing system(s) or are the existing system(s) to be replaced entirely?

1. Impact Analysis

The impact analysis assesses the impacts of the system within its operational setting. Both positive impacts (e.g., increased productivity, higher product sales) as well as negative impacts (e.g., job displacement, potential negative legal impacts) are addressed.

1. Related Systems

Related systems are those computer systems in use elsewhere that provide functionality similar or related to the functionality of the system being specified here.

Both commercially available and public domain systems are considered and the following issues are addressed:

- What is good about the related system, i.e., what features it has that should be included in the system being proposed here.
- What is bad about the related system, i.e., what features should not be included in the proposed system, or what features should be included but done in a different or better way.
- What is missing, i.e., what new features should be included in the proposed system that are not found in the related system
- Any other noteworthy features.

10.4. Summary

By now you should have some basics in an OOP programming language e.g Java or C++. Although it is not our aim to write complete Java codes in this unit, you should be able to translate UML diagrams to java code and even Java code to UML diagrams.

Learning Activity I

Define a Java program to implement the following UML diagram. Note that the types have not been indicated in the diagram but you should indicate them in the class definition.

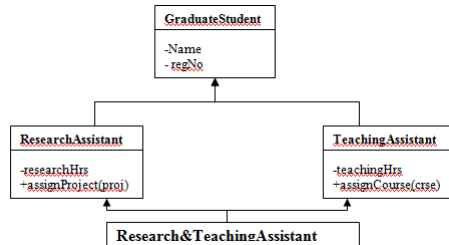


Figure 10.10: UML Specification

Learning Activity II

Below is a JAVA program which implements an Employee system definition. Deduce a UML diagram from the code, clearly giving the attributes and types of each class.

```
public class Employee {
    private String empName; private Int EmpNumber; private String Address; private
    String DateHired;
    Employee(String en, Int ENo, String Ad, String dh)
    { empName = en; EmpNumber = ENo; Address = Ad; DateHired = dh; }
    void getName() { }
    void getNumber() {}
    void getDateHired() {}
    void getAddress() {} }
    Class HourlyEmployee extends Employee {
    private double hourlyRate; private double hours;
    HourlyEmployee(String en, Int ENo, String Ad, String dh, double hR, double Hr)
    {
    Super(en, ENo, Ad, dh); hourlyRate = hR; hours = Hr; }
    Double ComputeWages() {
    return hourlyRate * hours; }
    }
    Class SalariedEmployee extends Employee {
```

```
private double AnnualSalary; SalariedEmployee(String en, Int ENo, String Ad,
String dh, double As) {
Super(en, ENo, Ad, dh); AnnualSalary = As; }
void setAnnualSalary() { }
}
class Consultant extends Employee {
private String ContractNumber; private double billingRate; private double hours;
Consultant(String en, Int ENo, String Ad, String dh, String CN, double Br, double
hr) {
Super(en, ENo, Ad, dh); billingRate= Br; hours = hr; }
double ComputeFee() {
return billingRate * hours;
}
}
```

Learning Activity III

A bank is finally ready to create a new fully-computerized management system! The system to be developed should enable users to create bank accounts. Bank accounts maintain their own balance. Customers should be able to deposit and withdraw money to/from their accounts, by visiting a bank teller in one of the branches. Customers should also be able to transfer funds from one bank account to another (in other words, the money should be withdrawn from the source account, and deposited into the destination account). The Bank manager should also calculate the interest accumulated each month for each account, and deposit that amount into the account. Also, the bank manager should also calculate the monthly charges each month (for each account), and withdraw that amount from the account.

Use Case Determination

1. List the three actors present in the Bank's system.
2. List the six use cases present in the Bank's system. Hint: Do not list use cases not mentioned in the specification.
3. Draw the completed Use Case Diagram showing all 3 actors and all 6 use cases, as well as any relationships that might exist.

Class Identification

1. List the entity classes present in the Bank's System. Hint: Entity classes are classes that represent data or tangible items that the system would need to represent in some way.
2. Think of what attributes would be needed to support the Bank's System. Hint: Do not think about attributes not mentioned in the specification.
3. Draw a class diagram containing all classes listed in (1) and all attributes listed in (2), and also include any class relationships that might exist.

Learning Activities

1. Most modern development methods are based on an object-oriented approach to systems development.
 - Discuss why it is important for an object-oriented development method to be architecture-centric?
 - What does it mean for an object-oriented development method to be iterative and incremental?
 - Describe how the Unified Modeling Language (UML) diagrams support the USDP and Object Oriented software development in general
1. Produce a statechart diagram for the Bank Account class described below. Your diagram should include any guard conditions related to transitions between states:

A Bank Account can be in credit, debit or have a balance of zero. The events that affect these states are Charge (when money is paid into the account), Payment (when money is taken out of the account) and Close (when the customer closes the account). When an account is opened it has a balance of zero. From that point the account will move between the states indicated above depending on the value of payments and charges made.

Revision Questions

Example . The Unified software development process identifies four phases for software development. Explain each of the phases indicating the workflows (activities) associated with each.

Solution: for revision



EXERCISE 20.  Rational Unified Process (RUP) is a use case driven, architectural centric and iterative and incremental process. Explain the significance of use case driven, architectural centric and iterative and incremental process.

EXERCISE 21.  The Systems Development Life Cycle (SDLC) has had an enormous influence as a generalized method for developing systems. a) Distinguish between techniques, tools and methodologies in the context of the SDLC. b) What are the strengths and limitations of the traditional SDLC? c) Would you adopt the SDLC for developing a small database application on your stand-alone PC? Would you modify the SDLC in any way? Justify your answer.

References and Additional Reading Materials

1. Richard Lee , William M.Tepfenhart (2000). UML and C++ - A practical guide to object oriented development” , ISBN-13: 978-0130290403
2. John. W Satzinger (2004). Object Oriented Analysis and Design with the Unified Process. Cengage Learning, ISBN-13: 978-0619216436
3. Journal of Science and Technology ISSN 1860-4749